Platforma AWS Infrastructure Intelligence – System CSPM do automatycznej walidacji zgodności infrastruktury chmurowej

(AWS Infrastructure Intelligence Platform – CSPM System for Automated Cloud Infrastructure Compliance Validation)

Krystian Cybul

Lublin 2026

STRESZCZENIE

Praca opisuje projekt i implementację AWS Infrastructure Intelligence – systemu CSPM (Cloud Security Posture Management) walidującego zgodność infrastruktury AWS z normami bezpieczeństwa. Celem było stworzenie narzędzia weryfikującego zgodność z czterema frameworkami jednocześnie (CIS AWS Foundations Benchmark, SOC 2 Type II, HIPAA Security Rule, PCI-DSS v4.0). Rozwiązania komercyjne pozycjonowane są w segmencie korporacyjnym z wysokimi barierami cenowymi; opracowana platforma jest open-source.

Praca weryfikuje trzy hipotezy badawcze dotyczące zunifikowanego mapowania wieloframeworkowego, architektury dwutrybowej dla CI/CD oraz użyteczności raportów Excel dla zespołów compliance.

Implementacja powstała w języku Python 3.11 z wykorzystaniem biblioteki boto3. Czterowarstwowa architektura (wejściowa, kolekcji, analizy, raportowania) umożliwia skanowanie dziewięciu usług AWS: EC2, EBS, S3, RDS, VPC, IAM, Lambda, Security Groups oraz Cost Explorer, a warstwa analizy implementuje 58 reguł bezpieczeństwa (opartych na CIS AWS Foundations Benchmark v6.0.0 oraz uzupełnionych o reguły best-practice), mapowanych na wspomniane frameworki. Wyniki są generowane w pięciu formatach: Excel (jedenaście arkuszy z formatowaniem warunkowym), PDF, HTML, JSON oraz wykresy. Dostęp do platformy zapewniają REST API (FastAPI) i interfejs wiersza poleceń (Click).

Opracowany wzorzec architektury dwutrybowej pozwala uruchamiać system zarówno z rzeczywistymi danymi AWS, jak i z danymi symulowanymi (tryb mock), co umożliwia integrację z pipeline'ami CI/CD bez dostępu do środowiska produkcyjnego. Testy w trybie mock na 89 zasobach w trzech regionach wykazały 73 niezgodności przy wskaźniku zgodności 68%. Testy jednostkowe uruchamiane są w pipeline'ie CI.

Kod źródłowy liczy około 17 659 linii w 43 plikach (w tym testy). Wśród możliwych rozszerzeń znajdują się wsparcie Azure/GCP oraz automatyczna remediacja.

Słowa kluczowe: CSPM, AWS, bezpieczeństwo chmury, zgodność, CIS Benchmark, SOC 2, HIPAA, PCI-DSS, Python, boto3, DevSecOps

ABSTRACT

This thesis describes the design and implementation of AWS Infrastructure Intelligence, a Cloud Security Posture Management (CSPM) system for automated compliance validation of AWS infrastructure against security standards. The goal was to build an open-source tool that verifies compliance with four frameworks simultaneously: CIS AWS Foundations Benchmark, SOC 2 Type II, HIPAA Security Rule, and PCI-DSS v4.0. Commercial alternatives are positioned in the enterprise segment with high cost barriers.

Three research hypotheses are verified, concerning unified multi-framework mapping, dual-mode architecture for CI/CD integration, and the usefulness of Excel reports for compliance teams.

The system was implemented in Python 3.11 using the boto3 library. Its four-layer architecture (input, collection, analysis, reporting) scans nine AWS services: EC2, EBS, S3, RDS, VPC, IAM, Lambda, Security Groups, and Cost Explorer. The analysis layer applies 58 security rules (based on CIS AWS Foundations Benchmark v6.0.0 and supplemented with best-practice rules) mapped to the compliance frameworks listed above, and the reporting module exports results in five formats: Excel (eleven worksheets with conditional formatting), PDF, HTML, JSON, and charts. Access is provided through a REST API (FastAPI) and a command-line interface (Click).

A dual-mode architecture pattern allows the system to run with real AWS data or simulated data (mock mode), enabling CI/CD pipeline integration without production environment access. Tests in mock mode on 89 AWS resources across three regions detected 73 compliance violations with a 68% compliance score. Unit tests run as part of the CI pipeline.

The source code totals approximately 17,659 lines across 43 files (including tests). Possible extensions include Azure/GCP support and automated remediation.

Keywords: CSPM, AWS, cloud security, compliance, CIS Benchmark, SOC 2, HIPAA, PCI-DSS, Python, boto3, DevSecOps

SPIS TREŚCI

Streszczenie **1**

Abstract **2**

1 Wstęp **7**

1.1 Wprowadzenie do problematyki 7

1.2 Uzasadnienie wyboru tematu 7

1.3 Cele pracy 8

1.4 Hipotezy badawcze 9

1.5 Zakres pracy 9

1.6 Metodyka badawcza 10

1.7 Wkład własny autora 10

1.8 Struktura pracy 11

2 Podstawy teoretyczne **12**

2.1 Cloud Security Posture Management 12

2.1.1 Definicja i geneza CSPM 12

2.1.2 Funkcje systemów CSPM 12

2.1.3 Rynek narzędzi CSPM 13

2.1.4 Luka rynkowa 14

2.2 Frameworki zgodności bezpieczeństwa 15

2.2.1 CIS AWS Foundations Benchmark 15

2.2.2 SOC 2 Type II 15

2.2.3 HIPAA Security Rule 16

2.2.4 PCI-DSS v4.0 16

2.2.5 Porównanie frameworków 17

2.3 Usługi Amazon Web Services 18

2.3.1 Model współdzielonej odpowiedzialności 18

2.3.2 Usługi objęte skanowaniem 18

2.3.3 API AWS i biblioteka boto3 20

2.4 DevSecOps i CI/CD 21

2.4.1 Paradygmat DevSecOps 21

2.4.2 Ciągła integracja i ciągłe wdrażanie 21

2.4.3 GitHub Actions 21

2.5 Przegląd literatury 22

3 Metodologia i projekt systemu 23

3.1 Specyfikacja wymagań 23

3.1.1 Wymagania funkcjonalne 23

3.1.2 Wymagania niefunkcjonalne 23

3.2 Architektura systemu 24

3.2.1 Architektura wysokopoziomowa 24

3.2.2 Diagram komponentów 25

3.2.3 Przepływ danych 27

3.2.4 Wzorzec architektury dwutrybowej 28

3.3 Model danych 29

3.3.1 Schema JSON dla zasobów 29

3.3.2 Schema dla wyników analizy 29

3.3.3 Mapowanie reguł na frameworki 30

3.4 Stack technologiczny 30

3.4.1 Uzasadnienie wyboru technologii 30

3.4.2 Rozważane alternatywy 32

3.4.3 Zależności projektu 32

3.5 Środowisko testowe 32

3.5.1 Konfiguracja środowiska testowego (mock) 32

3.5.2 Środowisko developerskie 32

3.5.3 Dane testowe (tryb mock) 32

4 Implementacja i wyniki 34

4.1 Warstwa kolekcji danych (Collection Layer) 34

4.1.1 Abstrakcyjna klasa bazowa BaseCollector 34

4.1.2 Implementacja kolektorów usług 34

4.1.3 Obsługa paginacji API AWS 36

4.2 Warstwa analizy zgodności (Analysis Layer) 36

4.2.1 ComplianceAnalyzer – rdzeń systemu 36

4.2.2 Algorytm oceny zgodności 36

4.2.3 Moduły frameworków zgodności 36

4.3 Warstwa raportowania (Reporting Layer) 37

4.3.1 ExcelReporter – raport korporacyjny 37

4.3.2 ChartGenerator – wizualizacje 38

4.4 REST API 39

4.4.1 Architektura API 39
4.4.2 Dokumentacja OpenAPI 39
4.5 Pipeline CI/CD 40
4.5.1 Struktura przepływu pracy w GitHub Actions 40
4.5.2 Etapy pipeline'u 40
4.6 Wyniki testów 41
4.6.1 Testy automatyczne 41
4.6.2 Wyniki skanowania środowiska testowego 42
4.6.3 Wyniki skanowania w rozbiciu na usługi 43
4.6.4 Uwagi dotyczące walidacji 44
4.7 Wkład własny autora 44
4.7.1 Zunifikowany mapper wieloframeworkowy 44
4.7.2 Wzorzec architektury dwutrybowej 44
4.7.3 Pakiet raportowania 44
4.8 Napotkane wyzwania techniczne 45
4.8.1 Obsługa paginacji boto3 45
4.8.2 Ograniczanie przepustowości API AWS 45
4.8.3 Obsługa polskich znaków w raportach PDF 46
4.8.4 Podsumowanie wyzwań 46

5 Zakończenie **47**

5.1 Podsumowanie pracy 47
5.2 Weryfikacja hipotez badawczych 48
5.3 Wkład inżynierski i praktyczny 49
5.3.1 Wkład inżynierski 49
5.3.2 Wkład praktyczny 49
5.4 Ograniczenia pracy 50
5.5 Kierunki dalszego rozwoju 51
5.6 Wnioski końcowe 52

Bibliografia **53**

Spis rysunków **56**

Spis tabel **57**

Spis listingów **58**

Załączniki **59**

A Wybrane fragmenty kodu źródłowego **59**

A.1 Klasa BaseCollector 59

A.2 Fragment ComplianceAnalyzer – reguły CIS 61

A.3 Struktura katalogów projektu 62

B Przykładowa lista wykrytych niezgodności **64**

B.1 Niezgodności w domenie IAM 64

B.2 Niezgodności w domenie Storage 65

B.3 Niezgodności w domenie Compute 65

B.4 Niezgodności w domenie Networking 66

B.5 Niezgodności w domenie Logging 66

B.6 Podsumowanie 66

ROZDZIAŁ 1

WSTĘP

1.1 Wprowadzenie do problematyki

Usługi chmurowe zmieniły sposób, w jaki organizacje budują i utrzymują infrastrukturę IT. Badanie CNCF Annual Survey 2024 (publ. 2025) wskazuje, że podejście cloud-native stało się powszechne, a ok. jedna czwarta respondentów deklaruje, że niemal cała ich praca developerska i wdrożeniowa wykorzystuje techniki cloud-native [13]. Amazon Web Services utrzymuje pozycję lidera rynku IaaS/PaaS (stan na Q3 2025) [32]. Migracja do chmury stworzyła nowe problemy bezpieczeństwa, których tradycyjne narzędzia ochrony infrastruktury on-premises nie rozwiązują.

Błędna konfiguracja zasobów chmurowych (ang. *cloud misconfiguration*) jest jednym z głównych wektorów naruszeń bezpieczeństwa. Cloud Security Alliance klasyfikuje błędną konfigurację jako jedno z głównych zagrożeń w chmurze publicznej [14]. Publiczne buckety S3, otwarte porty w Security Groups, nieszyfrowane wolumeny EBS, nierotowane klucze IAM – to typowe podatności prowadzące do wycieków danych.

Dlatego powstała kategoria narzędzi Cloud Security Posture Management (CSPM, pol. zarządzanie stanem bezpieczeństwa chmury) – systemów automatycznie oceniających konfigurację i dostarczających rekomendacje poprawy postawy bezpieczeństwa w środowiskach chmurowych [24].

1.2 Uzasadnienie wyboru tematu

Rynek narzędzi CSPM w analizowanych źródłach jest silnie zorientowany na rozwiązania komercyjne o wysokich barierach cenowych. Security Hub w modelu CSPM rozlicza się w podejściu opartym na jednostkach zasobów (Resource Units), gdzie stawka zależy od typu i liczby monitorowanych zasobów (stan na 2026, region us-east-1) [6]. Przy dużej skali model ten generuje zauważalne koszty. Prisma Cloud firmy Palo Alto Networks oferuje rozbudowaną platformę, a szczegóły oferty zależą od skali wdrożenia; w przeglądzie publicznej dokumentacji (2023–2026) nie znaleziono jednolitego cennika. Wiz – startup założony w 2020 roku, notowany w zestawieniu Forbes Cloud 100 po rundzie finansowania ogłoszonej w 2024 roku [17] – jest przykładem komercyjnego rozwiązania CSPM.

Istniejące rozwiązania open-source, takie jak Prowler czy ScoutSuite, częściowo wypełniają tę lukę, ale mają ograniczenia w zakresie raportowania. Prowler wspiera wiele frameworków zgodności (CIS, SOC 2, HIPAA, PCI-DSS i inne), a domyślne formaty wyjściowe to CSV, JSON i HTML; w przeglądzie dokumentacji i repozytorium (2023–2026) nie znaleziono potwierdzenia natywnego eksportu do wieloarkuszowego

Excela z formatowaniem warunkowym ani do PDF [30]. ScoutSuite skanuje szeroki zakres usług, ale generuje raport HTML, a dane przechowuje w pliku JSON, co utrudnia integrację z procesami audytowymi wymagającymi formatów korporacyjnych [27].

Pracując jako inżynier DevOps w firmie z sektora finansowego, autor doświadczył tej luki. Organizacja potrzebowała narzędzia walidującego zgodność z CIS Benchmark (wymaganie wewnętrzne), SOC 2 (audyt zewnętrzny), HIPAA (dane medyczne) oraz PCI-DSS (płatności), generującego raporty w formatach korporacyjnych, które w praktyce są wygodne dla zespołów audytu i compliance. Jak opisano powyżej, dostępne narzędzia open-source nie oferują natywnego wieloarkuszowego raportu Excel ani eksportu do PDF, a budżet wykluczał rozwiązania komercyjne. Stąd pomysł na niniejszą pracę.

1.3 Cele pracy

Celem pracy jest zaprojektowanie i implementacja platformy do automatycznej walidacji infrastruktury AWS (Cloud Security Posture Management, CSPM) względem wielu frameworków zgodności. Cele szczegółowe:

1. **Cel poznawczy**: Przeprowadzenie analizy porównawczej istniejących narzędzi typu CSPM oraz identyfikacja luk funkcjonalnych, które uzasadniają stworzenie nowego rozwiązania.

2. **Cel projektowy**: Opracowanie architektury systemu umożliwiającej modułowe skanowanie usług AWS, z naciskiem na rozszerzalność i łatwość dodawania nowych reguł zgodności.

3. **Cel implementacyjny**: Stworzenie działającej platformy w języku Python, obejmującej:

   - kolektory dla minimum 9 usług AWS (EC2, EBS, S3, RDS, VPC, IAM, Security Groups, Lambda, Cost Explorer),
   - mapowanie 58 reguł bezpieczeństwa (opartych na CIS AWS Foundations Benchmark v6.0.0 oraz uzupełnionych o reguły best-practice) na 4 frameworki (CIS, SOC 2, HIPAA, PCI-DSS),
   - generowanie raportów w 5 formatach (Excel, PDF, HTML, JSON, wykresy),
   - REST API (Representational State Transfer Application Programming Interface) oraz interfejs CLI (Command Line Interface, pol. interfejs wiersza poleceń).

4. **Cel walidacyjny**: Przeprowadzenie testów platformy w trybie symulowanym (mock) na reprezentatywnym zestawie zasobów AWS, z oceną dokładności wykrywania niezgodności.

1.4 Hipotezy badawcze

Na podstawie analizy problemu oraz przeglądu istniejących rozwiązań sformułowano następujące hipotezy badawcze:

H1: Można zaprojektować system CSPM walidujący zgodność z wieloma frameworkami (CIS, SOC 2, HIPAA, PCI-DSS) bez duplikacji logiki przez zastosowanie zunifikowanego mechanizmu mapowania reguł.

H2: Architektura dwutrybowa (produkcyjna/testowa) umożliwia integrację systemu CSPM z pipeline'ami CI/CD bez dostępu do środowiska produkcyjnego AWS.

H3: Raportowanie w formatach korporacyjnych (Excel z formatowaniem warunkowym) zwiększa użyteczność narzędzia CSPM dla zespołów compliance i audytu w porównaniu z raportami HTML/JSON.

Weryfikacja hipotez zostanie przedstawiona w rozdziale 5.

1.5 Zakres pracy

Zakres pracy obejmuje analizę wymagań, projektowanie architektury, implementację, testowanie oraz dokumentację. Platforma została ograniczona do środowiska Amazon Web Services, z uwzględnieniem możliwości przyszłego rozszerzenia na Azure i GCP.

W ramach pracy **zrealizowano**:

- architekturę 4-warstwową (wejściowa, kolekcji, analizy, raportowania),
- 9 kolektorów dla usług AWS,
- 58 reguł bezpieczeństwa mapowanych na 4 frameworki,
- 5 formatów raportów z formatowaniem profesjonalnym,
- REST API (FastAPI) oraz CLI (Click),
- tryb mock umożliwiający testy bez dostępu do AWS,
- pipeline CI/CD w GitHub Actions (6 etapów),
- konteneryzację Docker z wieloetapowym budowaniem,
- dokumentację techniczną i użytkownika.

Poza zakresem pracy pozostają:

- automatyczna remediacja wykrytych niezgodności,
- wsparcie dla dostawców innych niż AWS,
- interfejs webowy z panelem w czasie rzeczywistym,
- integracja z systemami SIEM/SOAR.

1.6 Metodyka badawcza

W pracy zastosowano połączenie metod badawczych charakterystycznych dla inżynierii oprogramowania:

Pracę rozpoczęto od przeglądu literatury i dokumentacji technicznej: publikacji o bezpieczeństwie chmury, dokumentacji AWS oraz specyfikacji frameworków (CIS, SOC 2, HIPAA, PCI-DSS). Wykorzystano źródła branżowe i akademickie, w większości z lat 2018–2026.

Następnie przeprowadzono analizę porównawczą istniejących narzędzi CSPM (AWS Security Hub, Prisma Cloud, Prowler, ScoutSuite), zestawiając je według kryteriów: zakres skanowania, wspierane frameworki, formaty raportów, model cenowy.

System implementowano iteracyjnie w trzech etapach:

1. MVP (minimalny produkt funkcjonalny) – podstawowe skanowanie EC2/S3 z raportem JSON,
2. wersja beta – pełen zestaw kolektorów i analizatorów,
3. wersja produkcyjna – optymalizacja wydajności, pełna dokumentacja.

Weryfikację prowadzono na wielu poziomach: testy jednostkowe (pytest), testy integracyjne (tryb mock) oraz testy wydajnościowe w trybie mock.

1.7 Wkład własny autora

Autor samodzielnie zaprojektował i zaimplementował platformę AWS Infrastructure Intelligence. Wkład własny obejmuje:

1. **Zunifikowany mapper wieloframeworkowy**: mechanizm mapowania reguł bezpieczeństwa na wiele frameworków jednocześnie, bez duplikacji kodu, zgodny ze specyfikacjami CIS, SOC 2, HIPAA i PCI-DSS.
2. **Wzorzec architektury dwutrybowej**, pozwalający przełączać się między trybem produkcyjnym (API AWS) a trybem mock (dane symulowane), co ułatwia integrację z pipeline'ami CI/CD.
3. **Pakiet raportowania** generujący raporty w formatach oczekiwanych przez zespoły audytu, w tym ExcelReporter z jedenastoma arkuszami i formatowaniem warunkowym.
4. **Wzorzec integracji CI/CD**: 6-etapowy pipeline GitHub Actions (lint, test, integracja, bezpieczeństwo, budowanie, dokumentacja).

Kod źródłowy liczy około 17 659 linii w 43 plikach (w tym testy). Projekt udostępniono jako open-source na licencji MIT.

1.8 Struktura pracy

Praca składa się z pięciu rozdziałów, bibliografii oraz dwóch załączników.

Rozdział 1 (niniejszy) wprowadza w problematykę bezpieczeństwa chmury, formułuje cele i metodykę.

Rozdział 2 zawiera podstawy teoretyczne: definicje CSPM, przegląd frameworków, charakterystykę usług AWS, wprowadzenie do DevSecOps i CI/CD.

Rozdział 3 przedstawia metodologię i projekt systemu: wymagania, architekturę, model danych, uzasadnienie wyboru technologii.

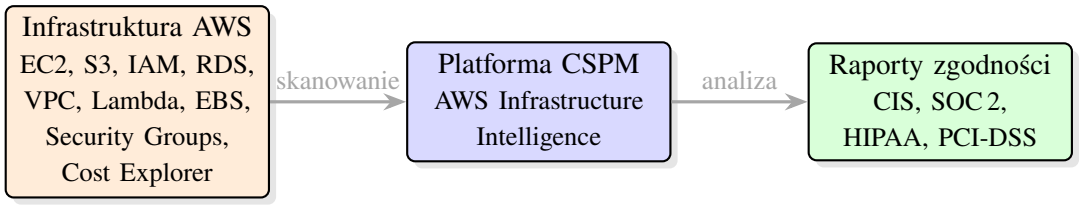
Rozdział 4 opisuje implementację, wyniki testów i napotkane wyzwania techniczne.

Rozdział 5 podsumowuje pracę i wskazuje kierunki dalszego rozwoju.

Załącznik A zawiera wybrane fragmenty kodu źródłowego platformy.

Załącznik B przedstawia wybrane wyniki skanowania w trybie mock (42 spośród 73 wykrytych niezgodności).

Kontekst problemu CSPM ilustruje rysunek 1.1.



Rys. 1.1. Kontekst problemu Cloud Security Posture Management

Źródło: Opracowanie własne

ROZDZIAŁ 2

PODSTAWY TEORETYCZNE

2.1 Cloud Security Posture Management

2.1.1 Definicja i geneza CSPM

Cloud Security Posture Management (CSPM) obejmuje narzędzia służące do oceny i poprawy postawy bezpieczeństwa w chmurze, w tym identyfikację niezgodnych konfiguracji oraz rekomendacje działań naprawczych [24].

CSPM powstało w odpowiedzi na rosnącą liczbę incydentów wynikających z błędów konfiguracyjnych. W modelu on-premise administrator kontrolował każdy serwer i firewall. Chmura odwróciła ten model: tysiące zasobów można utworzyć w minuty kilkoma kliknięciami lub liniami kodu Infrastructure as Code. Łatwość tworzenia zasobów nie została zrównoważona prostymi mechanizmami weryfikacji ich bezpieczeństwa [31].

Incydent Capital One z lipca 2019 roku uświadomił branży skalę problemu. Atakujący wykorzystał błędnie skonfigurowany WAF (Web Application Firewall) do przeprowadzenia ataku SSRF (Server-Side Request Forgery) na serwis metadanych EC2, a następnie – korzystając ze zbyt szeroko skonfigurowanej roli IAM – uzyskał dostęp do danych ponad 100 milionów klientów [9]. Kara nałożona przez Office of the Comptroller of the Currency wyniosła 80 milionów dolarów [28], a całkowite koszty incydentu (w tym remediacja, monitoring kredytowy i działania prawne) były istotne z perspektywy organizacji. Pokazało to, że okresowe audyty i manualne przeglądy konfiguracji nie nadążają za tempem zmian w środowiskach chmurowych.

2.1.2 Funkcje systemów CSPM

Platformy CSPM realizują kilka funkcji, z których najważniejszą jest ciągłe monitorowanie konfiguracji. Systemy odpytują API dostawcy chmury i pobierają aktualny stan zasobów, co pozwala wykryć niezgodności w ciągu minut od ich powstania – w odróżnieniu od audytów okresowych. Cloud Security Alliance wskazuje ograniczoną widoczność i observability jako jedno z głównych zagrożeń dla bezpieczeństwa chmury [14].

Zebrane dane wymagają interpretacji w kontekście standardów, dlatego CSPM tłumaczy parametry konfiguracji na wymagania frameworków zgodności. Pojedyncza reguła – np. „S3 bucket nie może mieć publicznego dostępu” – mapuje się jednocześnie na kontrole CIS, SOC 2, HIPAA i PCI-DSS, co eliminuje ręczne przeszukiwanie dokumentacji [11, 7, 33, 29].

Bardziej zaawansowane platformy wykraczają poza binarną klasyfikację (zgodny/niezgodny) i implementują ocenę ryzyka (ang. *risk scoring*): każdemu

znalezisku przypisują poziom krytyczności, a następnie obliczają zagregowany wskaźnik bezpieczeństwa. Scoring uwzględnia kontekst – publicznie dostępna baza z danymi osobowymi otrzymuje wyższą wagę niż identyczna baza w sieci prywatnej [16].

Częścią ekosystemu CSPM jest też automatyczna remediacja, czyli naprawianie niezgodności bez interwencji operatora. W praktyce wiąże się to z ryzykiem: wyłączenie publicznego dostępu do bucketa S3 może zablokować działającą aplikację. Organizacje często ograniczają auto-remediację do środowisk nieprodukcyjnych [25].

Odrębnym trendem jest integracja CSPM z pipeline'ami CI/CD, gdzie szablony IaC (Terraform, CloudFormation) są analizowane jeszcze przed wdrożeniem. Podejście „przesunięcia w lewo" (ang. *shift-left*) pozwala wykryć błędy konfiguracyjne na etapie przeglądu kodu, zanim trafią do środowiska produkcyjnego [8].

2.1.3 Rynek narzędzi CSPM

Rynek CSPM rozwija się wraz z rosnącą adopcją chmury i cloud-native [13]. Na rynku dostępne są zarówno platformy CNAPP, jak i wyspecjalizowane narzędzia CSPM. Przykładowi dostawcy komercyjni:

- **Palo Alto Networks Prisma Cloud** – platforma CNAPP (Cloud-Native Application Protection Platform) z modułami CSPM, CWPP (Cloud Workload Protection Platform), bezpieczeństwa kontenerów i serverless. Model cenowy oparty na kredytach; koszty zależą od skali wdrożenia.
- **Wiz** – startup założony w 2020 roku przez byłych pracowników Microsoft, notowany w zestawieniu Forbes Cloud 100 po rundzie finansowania ogłoszonej w 2024 roku [17]. Wyróżnia się skanowaniem bezagentowym i oceną ryzyka opartą na grafach.
- **AWS Security Hub** – natywna usługa Amazon integrująca wyniki z GuardDuty, Inspector, Macie i partnerów. Model cenowy oparty jest na jednostkach zasobów (Resource Units); w modelu rozliczania per-feature CSPM było rozliczane per-check [6].
- **Microsoft Defender for Cloud** – odpowiednik Security Hub dla Azure, z rosnącym wsparciem dla AWS i GCP [24].

Rozwiązania open-source są alternatywą dla organizacji z ograniczonym budżetem:

- **Prowler** – narzędzie CLI napisane w Pythonie, wspierające wiele frameworków zgodności (CIS, SOC 2, HIPAA, PCI-DSS, GDPR, NIST 800-53 i inne). Oferuje setki kontroli z eksportem do CSV, JSON i HTML, lecz nie posiada natywnego eksportu do wieloarkuszowego Excela z formatowaniem warunkowym ani raportów PDF (stan na 2026) [30].

- **ScoutSuite** – wielochmurowe narzędzie (AWS, Azure, GCP, Alibaba, Oracle) generujące interaktywne raporty HTML, a dane raportu przechowywane są w pliku w formacie JSON. Brak wsparcia dla eksportu do formatów korporacyjnych (Excel, PDF) [27].
- **Cloud Custodian** – silnik reguł oparty na YAML, pozwalający definiować własne polityki bezpieczeństwa. Konfiguracja wymaga znajomości filtrów i akcji specyficznych dla każdej usługi [12].

Porównanie wybranych narzędzi przedstawiono w tabeli 2.1.

Tab. 2.1. Porównanie wybranych narzędzi CSPM

Kryterium	**Prisma Cloud**	**AWS Security Hub**	**Prowler**	**AWS Infrastructure Intelligence***
Cena (rok)	Enterprise	Od kilku tys. USD	0 (OSS)	0 (OSS)
Frameworki	Wiele (zależne od oferty)	CIS, PCI-DSS i inne	CIS, SOC 2, HIPAA, PCI-DSS i inne	CIS, SOC 2, HIPAA, PCI-DSS
Formaty raportów	Panel, PDF	Panel, JSON	CSV, HTML, JSON	Excel, PDF, HTML, JSON, wykresy
Wiele chmur	Tak	Nie	Tak (wg dokumentacji)	Nie (AWS)
Integracja CI/CD	Tak	Ograniczona	Tak	Tak (mock)
REST API	Tak	Tak	Nie (CLI)	Tak

Źródło: Opracowanie własne na podstawie dokumentacji producentów

* AWS Infrastructure Intelligence – platforma będąca przedmiotem niniejszej pracy

2.1.4 Luka rynkowa

Analiza rynku wskazuje lukę w zakresie raportowania. Prowler generuje wyniki w formatach CSV, JSON i HTML – dane CSV można przekształcić do Excela zewnętrznymi narzędziami, lecz nie jest to natywny eksport z formatowaniem. ScoutSuite udostępnia interaktywne raporty HTML oraz surowe dane JSON, bez możliwości eksportu do formatów korporacyjnych. Żadne z analizowanych darmowych narzędzi (stan na 2026) nie łączy wieloframeworkowej walidacji z natywnym wieloarkuszowym raportem Excel i raportami PDF z wykresami.

Platforma opisana w niniejszej pracy wypełnia tę lukę: waliduje zgodność z wieloma frameworkami, generuje raporty w pięciu formatach (Excel, PDF, HTML, JSON, wykresy) i obsługuje tryb mock dla CI/CD.

2.2 Frameworki zgodności bezpieczeństwa

2.2.1 CIS AWS Foundations Benchmark

Center for Internet Security (CIS) to organizacja non-profit założona w 2000 roku, tworząca praktyki bezpieczeństwa informatycznego. CIS Controls (dawniej SANS Top 20) i CIS Benchmarks są powszechnie stosowane jako punkt odniesienia dla konfiguracji systemów operacyjnych, baz danych i środowisk chmurowych [10].

Niniejsza praca opiera się na CIS AWS Foundations Benchmark, obejmującym zestaw kontroli podzielonych na pięć domen[1]:

1. **Identity and Access Management** – wymuszanie MFA dla użytkowników root i IAM, rotacja kluczy dostępu co 90 dni, zasada najmniejszych uprawnień, eliminacja nieużywanych credentials.

2. **Storage** – szyfrowanie S3 server-side, blokowanie publicznego dostępu, włączenie wersjonowania, logowanie dostępu do bucketów.

3. **Logging** – włączenie CloudTrail we wszystkich regionach, walidacja integralności logów, szyfrowanie logów KMS, retencja minimum 90 dni.

4. **Monitoring** – alarmy CloudWatch dla operacji krytycznych (zmiany IAM, logowania root, modyfikacje VPC), metryki dla nieautoryzowanych wywołań API.

5. **Networking** – eliminacja otwartych Security Groups (0.0.0.0/0 na portach 22, 3389), domyślne Security Groups bez reguł, VPC Flow Logs.

Każda kontrola posiada unikalny identyfikator, poziom CIS (Level 1 – podstawowy, Level 2 – zaawansowany), uzasadnienie oraz procedurę weryfikacji manualnej i automatycznej. Przykładowa kontrola CIS wymaga, aby żaden użytkownik IAM nie posiadał kluczy dostępu starszych niż 90 dni [11].

2.2.2 SOC 2 Type II

SOC 2 (Service Organization Control 2) to framework audytowy opracowany przez American Institute of Certified Public Accountants (AICPA). W odróżnieniu od CIS Benchmark, który definiuje konkretne parametry techniczne, SOC 2 określa pięć Trust Service Criteria (TSC) na wyższym poziomie abstrakcji [7]:

1. **Security** (wspólne kryterium CC) – ochrona przed nieautoryzowanym dostępem, w tym kontrola dostępu, szyfrowanie, monitoring.

[1] W momencie przygotowania pracy dostępna była wersja v6.0.0. Implementacja platformy opiera się na tej wersji benchmarku.

2. **Availability** – zapewnienie dostępności systemów zgodnie z SLA, w tym redundancja, disaster recovery, capacity planning.

3. **Processing Integrity** – kompletność i poprawność przetwarzania danych, w tym walidacja wejść, kontrola jakości, reconciliation.

4. **Confidentiality** – ochrona informacji oznaczonych jako poufne, w tym szyfrowanie at-rest i in-transit, klasyfikacja danych.

5. **Privacy** – zgodność z polityką prywatności organizacji w zakresie zbierania, przetwarzania i usuwania danych osobowych.

Audyt SOC 2 Type II (w odróżnieniu od Type I) weryfikuje nie tylko projekt kontroli, ale również ich skuteczność operacyjną w określonym okresie (zwykle kilka miesięcy). Raport SOC 2 jest powszechnie wymagany przez klientów korporacyjnych przed nawiązaniem współpracy z dostawcą SaaS [7].

Mapowanie kontroli technicznych AWS na kryteria SOC 2 wymaga interpretacji. Przykładowo, wymaganie „szyfrowanie S3 server-side" realizuje zarówno kryterium Security (CC6.1 – ochrona danych), jak i Confidentiality (C1.1 – ochrona informacji poufnych). Platforma AWS Infrastructure Intelligence implementuje takie mapowanie poprzez moduł `analyzers/frameworks/soc2.py`.

2.2.3 HIPAA Security Rule

Health Insurance Portability and Accountability Act (HIPAA) z 1996 roku to amerykańska regulacja federalna ustanawiająca standardy ochrony informacji zdrowotnych (Protected Health Information, PHI). HIPAA Security Rule (45 CFR Part 160 i 164) definiuje wymagania techniczne, administracyjne i fizyczne dla systemów przetwarzających ePHI (electronic PHI) [33].

Struktura HIPAA Security Rule obejmuje trzy kategorie zabezpieczeń: Administrative Safeguards (§164.308) – zarządzanie bezpieczeństwem, szkolenia, procedury incydentów; Physical Safeguards (§164.310) – kontrola dostępu fizycznego, bezpieczeństwo stacji roboczych; Technical Safeguards (§164.312) – kontrola dostępu, audyt, integralność, szyfrowanie transmisji.

W kontekście AWS wymagania te przekładają się na: szyfrowanie EBS i RDS, CloudTrail z pełnym logowaniem, IAM z MFA. Platforma implementuje wybrane kontrole HIPAA w module `analyzers/frameworks/hipaa.py`.

2.2.4 PCI-DSS v4.0

Payment Card Industry Data Security Standard (PCI-DSS) to standard bezpieczeństwa ustanowiony przez konsorcjum PCI Security Standards Council, reprezentujące główne organizacje płatnicze (Visa, Mastercard, American Express, Discover, JCB).

Każdy podmiot przetwarzający, przechowujący lub transmitujący dane kart płatniczych podlega wymogom PCI-DSS [29].

Wersja 4.0 (marzec 2022) porządkuje wymagania bezpieczeństwa w obszarach takich jak budowa bezpiecznej sieci, ochrona danych kart, zarządzanie podatnościami, kontrola dostępu oraz monitoring i testy. PCI-DSS 4.0 wprowadza również „customized approach" jako alternatywę dla tradycyjnych kontroli [29].

2.2.5 Porównanie frameworków

Frameworki różnią się zakresem, poziomem szczegółowości i charakterem (techniczny vs. procesowy). Tabela 2.2 przedstawia porównanie wybranych charakterystyk.

Tab. 2.2. Porównanie frameworków zgodności bezpieczeństwa

Aspekt	**CIS**	**SOC 2**	**HIPAA**	**PCI-DSS**
Typ	Techniczny benchmark	Framework audytowy	Regulacja federalna	Standard branżowy
Obowiązkowość	Dobrowolny	Dobrowolny	Obowiązkowy (USA, PHI)	Obowiązkowy (płatności)
Poziom szczegółowości	Wysoki (parametry)	Niski (cele)	Średni	Średnio-wysoki
Liczba kontroli	Zależna od wersji benchmarku	Zależna od TSC	Zależna od zakresu Security Rule	Zależna od wersji i zakresu
Certyfikacja	Samoocena	Audyt CPA	Audyt OCR	Audyt QSA/ISA

Źródło: Opracowanie własne na podstawie dokumentacji frameworków

Tab. 2.3. Mapowanie frameworków zgodności na domeny kontroli technicznych AWS

Framework	**IAM**	**Szyfrowanie**	**Sieć**	**Logging**
CIS (benchmark)	IAM (1.x)	Szyfrowanie (2.x)	Sieć (5.x)	Logging (3.x)
SOC 2	CC6.1–CC6.3	C1.1–C1.2	CC6.6	CC7.1–CC7.2
HIPAA	§164.312(d)	§164.312(a)(1)	§164.312(e)(1)	§164.312(b)
PCI-DSS	Req. 7–8	Req. 3–4	Req. 1–2	Req. 10

Źródło: Opracowanie własne na podstawie dokumentacji frameworków

Tabela 2.3 ilustruje, w jaki sposób cztery frameworki zgodności odnoszą się do tych samych domen kontroli technicznych (IAM, szyfrowanie, sieć, logowanie), lecz przy użyciu różnych identyfikatorów i poziomów szczegółowości. Platforma mapuje jedną regułę techniczną (np. „S3 bucket musi być zaszyfrowany") na odpowiednie wymagania we wszystkich frameworkach jednocześnie. Numeracja kontroli CIS w pracy odzwierciedla mapowanie użyte w implementacji narzędzia.

2.3 Usługi Amazon Web Services

2.3.1 Model współdzielonej odpowiedzialności

Amazon Web Services stosuje model współdzielonej odpowiedzialności (Shared Responsibility Model), który precyzyjnie rozgranicza obowiązki dostawcy chmury i klienta w zakresie bezpieczeństwa [4].

Odpowiedzialność AWS („Security OF the Cloud”):

- Bezpieczeństwo fizyczne centrów danych
- Infrastruktura sieciowa i sprzętowa
- Hypervisor i warstwa wirtualizacji
- Oprogramowanie usług zarządzanych (RDS engine, Lambda runtime)

Odpowiedzialność klienta („Security IN the Cloud”):

- Konfiguracja usług (Security Groups, IAM policies, S3 bucket policies)
- Szyfrowanie danych (at-rest, in-transit)
- Zarządzanie tożsamością i dostępem
- Patching systemu operacyjnego (EC2 instances)
- Konfiguracja aplikacji i zabezpieczenie kodu

Granica odpowiedzialności zależy od typu usługi. W przypadku EC2 (IaaS) klient odpowiada za cały stack od systemu operacyjnego w górę. W przypadku Lambda (FaaS) klient odpowiada wyłącznie za kod funkcji i jej konfigurację.

Systemy CSPM operują wyłącznie w obszarze odpowiedzialności klienta – weryfikują poprawność konfiguracji usług, nie mając wglądu w infrastrukturę AWS.

2.3.2 Usługi objęte skanowaniem

Platforma AWS Infrastructure Intelligence obejmuje dziewięć usług AWS. Wybór podyktowany był pokryciem kontroli wymaganych przez CIS AWS Foundations Benchmark (IAM, Storage, Logging, Networking), obecnością usług stanowiących podstawę większości wdrożeń (EC2, S3, RDS, VPC) oraz dostępnością danych kosztowych (Cost Explorer) do identyfikacji nieużywanych zasobów. W zestawie mock wybrane usługi stanowią większość zasobów, a kontrole globalne (np. CloudTrail, Config) są skanowane pośrednio. Poniżej opisano poszczególne usługi.

Amazon EC2 (Elastic Compute Cloud) to podstawowa usługa obliczeniowa AWS, umożliwiająca uruchamianie maszyn wirtualnych. Kolektor `EC2Collector` pobiera

informacje o instancjach, typach (t2.micro, m5.large itp.), AMI, statusie monitoringu szczegółowego oraz przypisanych Security Groups. Wśród typowych niezgodności znajdują się brak detailed monitoring, wykorzystanie przestarzałych AMI czy instancje bez profilu IAM.

Z EC2 powiązana jest usługa **Amazon EBS** (Elastic Block Store), dostarczająca wolumeny blokowe. `EBSCollector` zbiera dane o wolumenach, snapshotach, statusie szyfrowania i stanie przypisania (attached/unattached). Platforma weryfikuje tu przede wszystkim szyfrowanie wolumenów, obecność nieprzypisanych wolumenów generujących koszty oraz rotację snapshotów.

Inaczej wygląda skanowanie **Amazon S3** (Simple Storage Service), gdzie złożoność konfiguracji uprawnień dostępu sprawia, że jest to jedno z najczęstszych źródeł wycieków danych w chmurze AWS [14]. `S3Collector` analizuje buckety pod kątem publicznego dostępu, wymuszania HTTPS, wersjonowania, logowania dostępu, szyfrowania server-side oraz polityk bucket policies.

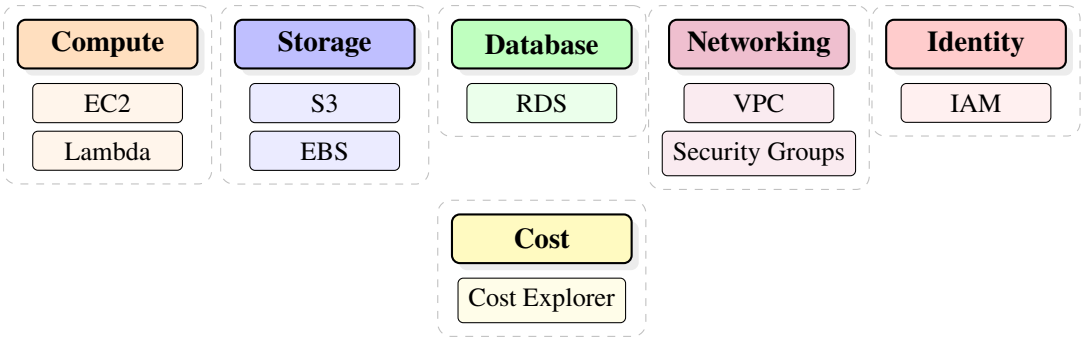
W przypadku **Amazon RDS** (Relational Database Service) platforma skupia się na zarządzanych bazach danych (MySQL, PostgreSQL, Oracle, SQL Server, Aurora). `RDSCollector` sprawdza szyfrowanie at-rest, publiczną dostępność, retencję backupów, konfigurację Multi-AZ oraz automatyczne minor version upgrades.

Warstwa sieciowa reprezentowana jest przez **Amazon VPC** (Virtual Private Cloud) i **Security Groups**. `VPCCollector` mapuje VPC, subnety, tablice routingu, Internet Gateways i NAT Gateways, a następnie weryfikuje obecność VPC Flow Logs, użycie domyślnych VPC (zalecane usunięcie) oraz poprawność routingu do Internet Gateway. `SecurityGroupCollector` z kolei wykrywa otwarte porty (0.0.0.0/0 na SSH/RDP) oraz nadmiarowe reguły ingress/egress.

Spośród wszystkich skanowanych usług **AWS IAM** (Identity and Access Management) obejmuje najwięcej kontroli CIS. `IAMCollector` jest najbardziej rozbudowanym kolektorem i analizuje użytkowników, role, polityki, grupy, klucze dostępu oraz certyfikaty MFA.

Uzupełnieniem zestawu są **AWS Lambda** i **AWS Cost Explorer**. W przypadku Lambda platforma weryfikuje konfigurację VPC (funkcje powinny być wdrożone w VPC dla dostępu do zasobów prywatnych), obecność sekretów w zmiennych środowiskowych oraz aktualność runtime. Cost Explorer dostarcza natomiast danych kosztowych per usługa, na podstawie których platforma identyfikuje nieużywane zasoby generujące koszty (nieprzypisane wolumeny EBS, bezczynne instancje EC2).

Rysunek 2.1 przedstawia dziewięć usług AWS skanowanych przez platformę, pogrupowanych według kategorii funkcjonalnej.



Rys. 2.1. Usługi AWS objęte skanowaniem przez platformę
Źródło: Opracowanie własne

2.3.3 API AWS i biblioteka boto3

Amazon Web Services udostępnia programistyczne interfejsy (API) dla wszystkich usług. API oparte są na protokole REST z uwierzytelnianiem Signature Version 4 (SigV4). Każde żądanie zawiera nagłówki Authorization z podpisem HMAC-SHA256 obliczonym z klucza dostępu, regionu, usługi i payloadu żądania [5].

boto3 to oficjalny SDK (Software Development Kit) AWS dla języka Python, utrzymywany przez Amazon. Biblioteka abstrahuje szczegóły protokołu, oferując wysokopoziomowy interfejs obiektowy [3]:

```
ec2 = boto3.client('ec2', region_name='us-east-1')
for r in ec2.describe_instances()['Reservations']:
    for i in r['Instances']: print(i['InstanceId'], i['State']['
    Name'])
```

Listing 2.1. Użycie boto3 do listowania EC2
Źródło: Opracowanie własne

boto3 obsługuje automatycznie:

- Uwierzytelnianie (z pliku credentials, zmiennych środowiskowych, profilu IAM instancji)
- Ponawianie z wykładniczym opóźnieniem przy błędach przejściowych
- Paginację wyników (wiele API AWS limituje odpowiedź do ograniczonej liczby elementów)
- Regionalizację (endpoints specyficzne dla regionu)

Platforma AWS Infrastructure Intelligence wykorzystuje boto3 w wersji 1.34+ z konfiguracją projektu: timeout 30 sekund i maksymalnie 3 retries. Obsługa paginacji została zaimplementowana poprzez generatory, co pozwala efektywnie przetwarzać duże zbiory zasobów bez ładowania wszystkich do pamięci.

2.4 DevSecOps i CI/CD

2.4.1 Paradygmat DevSecOps

DevSecOps (Development, Security, Operations) rozszerza DevOps o bezpieczeństwo. W modelu waterfall zespół security przeglądał kod tuż przed wdrożeniem, często znajdując problemy wymagające kosztownych poprawek. DevSecOps promuje podejście „przesunięcia w lewo": integrację bezpieczeństwa na każdym etapie cyklu życia oprogramowania [8].

Do praktyk DevSecOps należą m.in.: definiowanie polityk bezpieczeństwa jako kodu (ang. *security as code*) w repozytorium Git, przeglądanego na równi z kodem produkcyjnym – przykładem jest Open Policy Agent (OPA) walidujący konfigurację Terraform przed wdrożeniem. Pipeline CI/CD automatycznie skanuje kod źródłowy (SAST), zależności (SCA) i infrastrukturę (CSPM). Zamiast okresowych audytów stosuje się ciągłą weryfikację zgodności z frameworkami. Analizę zagrożeń przeprowadza się już na etapie projektowania, zanim powstanie kod (ang. *threat modeling*).

2.4.2 Ciągła integracja i ciągłe wdrażanie

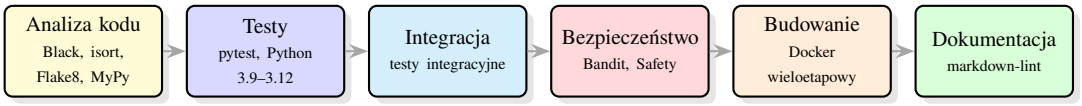
CI/CD to zestaw praktyk automatyzujących budowanie, testowanie i wdrażanie oprogramowania [21]:

Ciągła integracja (CI) – każda zmiana kodu inicjuje automatyczny proces budowania i testowania (checkout, instalacja zależności, linting, kompilacja, testy, skanowanie bezpieczeństwa). **Ciągłe wdrażanie (CD)** – automatyczne wdrażanie kodu do produkcji po pomyślnym przejściu pipeline'u CI.

2.4.3 GitHub Actions

GitHub Actions to platforma CI/CD zintegrowana z GitHub, umożliwiająca definiowanie workflow'ów jako plików YAML w katalogu `.github/workflows/`. Workflow składa się z zadań, które wykonują się równolegle (domyślnie) lub sekwencyjnie (z zależnościami `needs`) [20].

Kluczowe koncepty: workflow – plik YAML definiujący zautomatyzowany proces; zadanie – zestaw kroków wykonywanych na jednej maszynie; krok – pojedyncza operacja; akcja – wielokrotnie używalny komponent; wykonawca – maszyna, na której uruchamiane są zadania; artefakt – pliki wyjściowe; sekret – zaszyfrowana zmienna. Typową strukturę pipeline'u CI/CD przedstawia rysunek 2.2.



Rys. 2.2. Pipeline CI/CD platformy AWS Infrastructure Intelligence

Źródło: Opracowanie własne

Platforma AWS Infrastructure Intelligence implementuje 6-etapowy pipeline CI/CD, który zostanie szczegółowo omówiony w rozdziale 4.

2.5 Przegląd literatury

Bezpieczeństwo chmury jest przedmiotem badań od ponad dekady. W jednym z wczesnych przeglądów Subashini i Kavitha [31] zidentyfikowali 35 kategorii zagrożeń dla chmury publicznej, z błędną konfiguracją na trzecim miejscu. Rok później Zissis i Lekkas [34] zaproponowali wielowarstwowy model bezpieczeństwa, który nadal jest punktem odniesienia dla architektów systemów chmurowych. Modi et al. [25] rozwinęli ten kierunek, rekomendując automatyczne skanowanie i ciągły monitoring jako odpowiedź na dynamikę środowisk chmurowych. Fernandes et al. [16] wskazali lukę w automatyzacji weryfikacji zgodności, czemu poświęcona jest niniejsza praca. Ali et al. [1] przedstawili systematyczny przegląd szans i wyzwań bezpieczeństwa chmury. NIST CSF [26] definiuje pięć funkcji bezpieczeństwa: Identify, Protect, Detect, Respond, Recover.

Raporty branżowe: CNCF Annual Survey 2024 dokumentuje skalę adopcji cloud-native [13]; IBM X-Force 2025 opisuje dominujące wektory ataku, w tym nadużycia kont i eksploatację podatności [22]; ENISA Threat Landscape 2024 podsumowuje główne zagrożenia i trendy cyberbezpieczeństwa [15]; a CSA [14] identyfikuje główne zagrożenia chmurowe. Dokumentacja techniczna obejmuje: AWS Well-Architected Framework [2], CIS Benchmark [11], SOC 2 Guide [7], HIPAA Guidance [33], PCI-DSS [29].

Przegląd literatury i dokumentacji potwierdza, że istniejące narzędzia open-source (Prowler, ScoutSuite) eksportują dane do CSV, JSON i HTML. Uzyskanie formatów korporacyjnych (wieloarkuszowy Excel z formatowaniem warunkowym, PDF z wykresami) wymaga dodatkowej obróbki zewnętrznymi skryptami. Lukę tę szczegółowo omówiono w sekcji 2.1.4.

ROZDZIAŁ 3

METODOLOGIA I PROJEKT SYSTEMU

3.1 Specyfikacja wymagań

Specyfikację wymagań przeprowadzono zgodnie z metodyką IEEE 830 [23], uwzględniając trzy grupy użytkowników: inżynierów DevOps, zespół compliance i architektów bezpieczeństwa. Wywiady pozwoliły zidentyfikować 15 wymagań funkcjonalnych i 5 niefunkcjonalnych.

3.1.1 Wymagania funkcjonalne

Wymagania funkcjonalne zostały pogrupowane w cztery domeny: kolekcja danych, analiza zgodności, raportowanie oraz integracja.

Kolekcja danych (F1–F8): Pobieranie danych z 9 usług AWS (EC2, EBS, S3, RDS, VPC, IAM, Lambda, Security Groups, Cost Explorer) ze wszystkich wskazanych regionów. **F9**: Obsługa trybu mock dla testów.

Analiza zgodności (F10–F13): Walidacja względem CIS AWS Foundations Benchmark v6.0.0; mapowanie na SOC 2, HIPAA, PCI-DSS (łącznie 58 reguł bezpieczeństwa opartych na tej wersji benchmarku oraz uzupełnionych o reguły best-practice); obliczanie wskaźnika zgodności (0–100%); klasyfikacja krytyczności (Critical/High/Medium/Low/Info).

Raportowanie (F14–F15): Generowanie raportów Excel/PDF/HTML/JSON z wykresami.

3.1.2 Wymagania niefunkcjonalne

NF1 (Wydajność): Skanowanie 500 zasobów <5 min. **NF2** (Skalowalność): 3+ regionów jednocześnie. **NF3** (Niezawodność): Ponawianie z wykładniczym opóźnieniem. **NF4** (Pamięć): RAM <500 MB. **NF5** (Przenośność): Linux/macOS/Windows, Python 3.9+.

Macierz wymagań przedstawiono w tabeli 3.1.

Tab. 3.1. Macierz wymagań funkcjonalnych i niefunkcjonalnych

ID	Kategoria	Opis skrócony	Priorytet
F1–F8	Kolekcja	Pobieranie danych z 9 usług AWS	Wymagane
F9	Kolekcja	Tryb mock dla testów	Wymagane
F10	Analiza	Walidacja CIS Benchmark	Wymagane
F11	Analiza	Mapowanie na SOC 2/HIPAA/PCI-DSS	Zalecane
F12–F13	Analiza	Scoring i klasyfikacja	Wymagane
F14	Raportowanie	Formaty Excel/PDF/HTML/JSON	Wymagane
F15	Raportowanie	Generowanie wykresów	Zalecane
NF1	Wydajność	Czas skanowania <5 min	Wymagane
NF2	Skalowalność	Obsługa 3+ regionów	Wymagane
NF3	Niezawodność	Ponawianie z wykładniczym opóźnieniem	Wymagane
NF4	Pamięć	RAM <500 MB	Zalecane
NF5	Przenośność	Linux/macOS/Windows	Wymagane

Źródło: Opracowanie własne

3.2 Architektura systemu

3.2.1 Architektura wysokopoziomowa

Platforma AWS Infrastructure Intelligence opiera się na architekturze warstwowej z podziałem na cztery warstwy: wejściową, kolekcji, analizy i raportowania. Każda warstwa ma zdefiniowany interfejs i może być modyfikowana niezależnie od pozostałych [18].

Warstwa wejściowa (Input Layer) odpowiada za interakcję z użytkownikiem poprzez trzy kanały:

- CLI (Command Line Interface) – oparty na bibliotece Click, zapewniający dostęp z linii poleceń
- REST API – oparty na FastAPI, umożliwiający integrację z systemami zewnętrznymi
- Konfiguracja YAML – plik `config.yaml` definiujący parametry skanowania

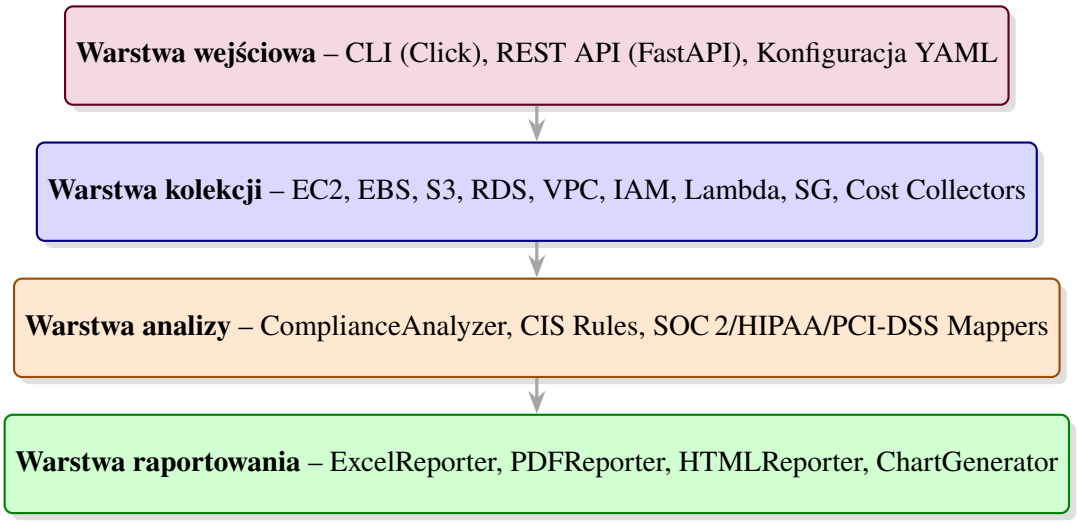
Warstwa kolekcji (Collection Layer) realizuje pobieranie danych z AWS poprzez dziewięć wyspecjalizowanych kolektorów. Każdy kolektor dziedziczy po klasie abstrakcyjnej `BaseCollector` i implementuje metodę `collect(regions: List[str]) -> Dict`. Warstwa obsługuje zarówno rzeczywiste API AWS (boto3), jak i dane symulowane (tryb mock).

Warstwa analizy (Analysis Layer) przetwarza zebrane dane, weryfikując zgodność z regułami bezpieczeństwa. Składa się z trzech analizatorów:

- `ComplianceAnalyzer` – walidacja CIS Benchmark i mapowanie na frameworki

- `SecurityAnalyzer` – ocena ryzyka i identyfikacja zagrożeń
- `CostOptimizer` – analiza kosztów i rekomendacje optymalizacyjne

Warstwa raportowania (Reporting Layer) transformuje wyniki analizy w raporty o formatach dostosowanych do różnych odbiorców. Pięć reporterów generuje: Excel, PDF, HTML, JSON oraz wykresy. Architekturę wysokopoziomową platformy przedstawia rysunek 3.1.



Rys. 3.1. Architektura wysokopoziomowa platformy AWS Infrastructure Intelligence
Źródło: Opracowanie własne

3.2.2 Diagram komponentów

Szczegółowa struktura komponentów systemu została przedstawiona na rysunku 3.2. Każdy prostokąt reprezentuje moduł Pythona (plik `.py`), a strzałki wskazują zależności importowe.

Moduł główny (`src/main.py`) zawiera klasę `InfrastructureScanner`, która orkiestruje cały proces skanowania. Klasa ta inicjalizuje konfigurację, tworzy instancje kolektorów i analizatorów, koordynuje przepływ danych oraz deleguje generowanie raportów.

Moduł CLI (`src/cli.py`) definiuje komendy dostępne z linii poleceń:

- `scan` – uruchomienie skanowania z parametrami
- `list-checks` – wyświetlenie listy kontroli z filtrowaniem
- `validate-config` – walidacja pliku konfiguracyjnego
- `test-connection` – test połączenia z AWS lub walidacja konfiguracji w trybie mock

Moduł API (`src/api/main.py`) implementuje REST API z endpointami:

- `POST /api/v1/scans` – inicjowanie skanowania asynchronicznego
- `GET /api/v1/scans/{id}` – pobieranie statusu/wyników skanowania
- `GET /api/v1/dashboard/summary` – dane dla panelu
- `GET /health` – kontrola stanu dla load balancerów

Pakiet kolektorów (`src/collectors/`) zawiera dziewięć modułów, każdy odpowiedzialny za jedną usługę AWS. Wspólna logika (obsługa paginacji, retry, logging) została wyekstrahowana do klasy bazowej `BaseCollector`.

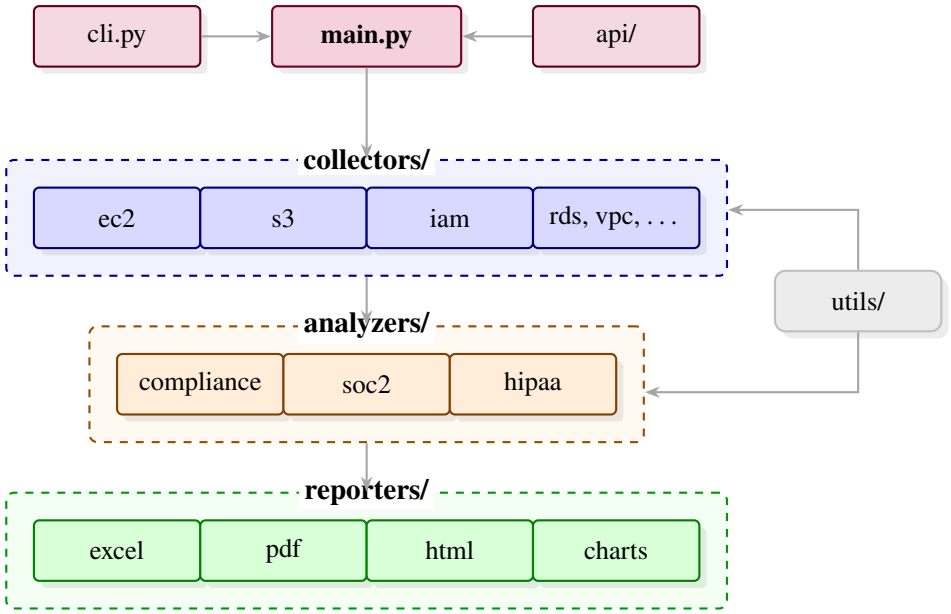
Pakiet analizatorów (`src/analyzers/`) grupuje moduły analizy:

- `compliance_analyzer.py` (2535 linii) – główny analizator z 58 regułami bezpieczeństwa
- `security_analyzer.py` (598 linii) – ocena ryzyka
- `cost_optimizer.py` (474 linii) – analiza kosztów
- `frameworks/` – moduły mapowania na SOC 2, HIPAA, PCI-DSS

Pakiet reporterów (`src/reporters/`) zawiera pięć generatorów raportów, każdy dostosowany do specyfiki formatu wyjściowego.

Pakiet utilities (`src/utils/`) dostarcza funkcje pomocnicze:

- `aws_session.py` – zarządzanie sesjami AWS, klient testowy
- `config_loader.py` – parsowanie YAML, walidacja schematu
- `logger.py` – konfiguracja loguru z rotacją plików



Rys. 3.2. Diagram komponentów platformy

Źródło: Opracowanie własne

3.2.3 Przepływ danych

Przepływ danych w systemie obejmuje pięć faz. Skanowanie rozpoczyna się od **inicjalizacji**: użytkownik uruchamia proces poprzez CLI, API lub bezpośrednio instancję `InfrastructureScanner`. System ładuje konfigurację z pliku YAML, waliduje parametry i inicjalizuje sesję AWS (lub klient mock w trybie mock).

Następnie, w fazie **kolekcji**, dla każdej włączonej usługi AWS tworzony jest odpowiedni kolektor. Kolektory wykonują się równolegle (domyślnie 5 wątków) w każdym regionie. Każdy kolektor zwraca słownik w ustandaryzowanym formacie:

```
{"resources": [...], "by_region": {"us-east-1": [...]}, "metadata":
    {...}}
```

Listing 3.1. Format wyniku kolektora

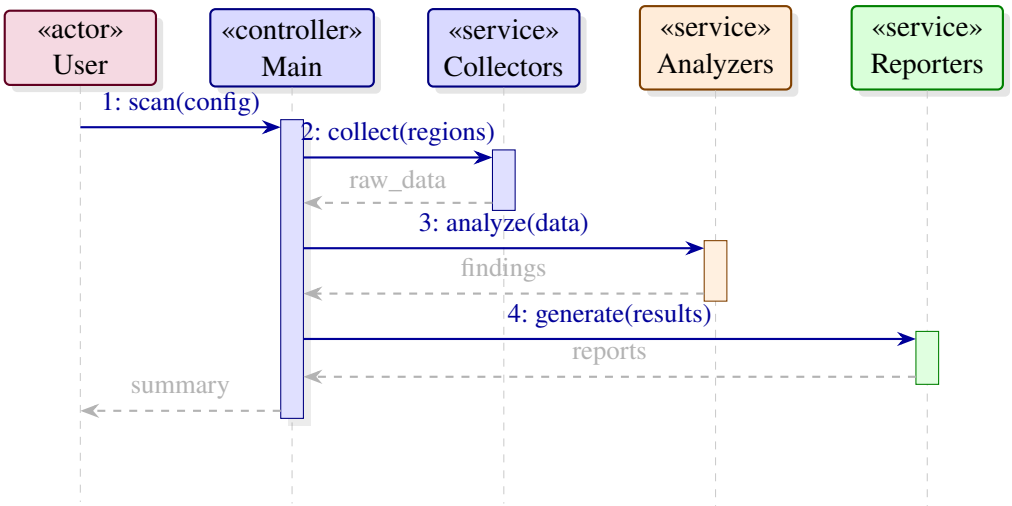
Źródło: Opracowanie własne

Zebrane dane trafiają do fazy **analizy**, gdzie są przekazywane sekwencyjnie do trzech analizatorów. `ComplianceAnalyzer` iteruje po wszystkich zasobach, weryfikując 58 reguł. Dla każdej niezgodności tworzona jest struktura `Finding` z polami: rule_id, severity, resource, description, remediation, framework_mappings.

W fazie **raportowania** wyniki analizy (zagregowane w strukturze `ScanResult`) trafiają do wybranych reporterów. Każdy reporter transformuje dane do swojego formatu i zapisuje plik wyjściowy.

Na końcu, w fazie **finalizacji**, system loguje podsumowanie (liczba zasobów, czas, znaleziska), zwraca wynik i kończy działanie (CLI) lub aktualizuje status zadania (API).

Przepływ danych ilustruje diagram sekwencji UML (rysunek 3.3).



Rys. 3.3. Diagram sekwencji UML procesu skanowania

Źródło: Opracowanie własne

3.2.4 Wzorzec architektury dwutrybowej

Wyzwaniem projektowym była obsługa dwóch trybów pracy: produkcyjnego (z API AWS) i mock (bez dostępu do AWS). Rozważano trzy podejścia:

Podejście 1: Warunkowe importy – sprawdzanie flagi `MOCK_MODE` w każdym kolektorze i warunkowe wywoływanie boto3 lub odczyt z pliku. Podejście to prowadziło do rozproszenia logiki i trudności w utrzymaniu.

Podejście 2: Wstrzykiwanie zależności – przekazywanie abstrakcyjnego interfejsu `AWSClient` do konstruktorów kolektorów. Wymagało znacznej refaktoryzacji i komplikowało inicjalizację.

Podejście 3: Wzorzec Factory z klientem testowym – pojedynczy punkt decyzyjny w klasie `AWSSessionManager`, który zwraca albo rzeczywisty klient boto3, albo obiekt `MockAWSClient`. Klient testowy implementuje identyczny interfejs, lecz zwraca dane z plików JSON.

Wybrano podejście trzecie ze względu na minimalne zmiany w kolektorach, centralizację logiki mock w jednym module i łatwość rozszerzania o nowe usługi.

```
1 class AWSSessionManager:
2     def get_client(self, svc_name: str, region: str):
3         if self.mock_mode:
4             return MockAWSClient(svc_name, self.mock_data_dir)
5         return boto3.client(svc_name, region_name=region)
```

Listing 3.2. Wzorzec architektury dwutrybowej

Źródło: Opracowanie własne na podstawie implementacji w `src/utils/aws_session.py`

3.3 Model danych

3.3.1 Schema JSON dla zasobów

Wszystkie zasoby AWS są reprezentowane w ustandaryzowanym formacie JSON, niezależnie od usługi źródłowej. Takie podejście ułatwia implementację analizatorów, które operują na wspólnej strukturze danych.

```
{"resource_id": "string", "resource_type": "enum[ec2_instance|
    s3_bucket|...]",
 "region": "string", "name": "string", "tags": {}, "properties":
    {}}
```

Listing 3.3. Schema JSON dla zasobu AWS
Źródło: Opracowanie własne

3.3.2 Schema dla wyników analizy

Wyniki analizy zgodności są reprezentowane przez strukturę `ComplianceResult`:

```
@dataclass
class ComplianceFinding:
    rule_id: str; severity: Severity; resource_id: str
    finding: str; remediation: str; framework_mappings: Dict

@dataclass
class ComplianceResult:
    overall_score: float; findings: List[ComplianceFinding]
    by_severity: Dict[Severity, int]; by_framework: Dict[str, float
    ]
```

Listing 3.4. Struktura wyników analizy
Źródło: Opracowanie własne na podstawie `src/analyzers/compliance_analyzer.py`

3.3.3 Mapowanie reguł na frameworki

Każda reguła CIS jest mapowana na odpowiadające kontrole w pozostałych frameworkach. Mapowanie przechowywane jest w słowniku statycznym, którego fragment przedstawiono poniżej:

Identyfikatory CIS w przykładzie poniżej odzwierciedlają mapowanie użyte w implementacji narzędzia.

```
FRAMEWORK_MAPPINGS = {
    "CIS-1.4": {"soc2": ["CC6.1"], "hipaa": ["164.312(d)"], "
    pci_dss": ["8.1.1"]},
    # ... 58 reguł z mapowaniem na 4 frameworki
}
```

Listing 3.5. Mapowanie reguł na frameworki
Źródło: Opracowanie własne

3.4 Stack technologiczny

3.4.1 Uzasadnienie wyboru technologii

Przy wyborze technologii kierowano się przede wszystkim stabilnością rozwiązań i kompatybilnością z ekosystemem AWS (oficjalny SDK boto3), a także produktywnością i aktywnością społeczności open-source. Tabela 3.2 przedstawia wybrane technologie.

Tab. 3.2. Stack technologiczny platformy z uzasadnieniem wyboru

Kategoria	**Technologia**	**Uzasadnienie**
Język	Python 3.11	Oficjalny SDK AWS (boto3), bogaty ekosystem bibliotek, wysoka produktywność, typowanie statyczne (type hints)
AWS SDK	boto3 1.34+	Oficjalny SDK, pełne pokrycie API AWS, automatyczne retry, paginatory
CLI Framework	Click 8.x	Deklaratywna definicja komend, automatyczna walidacja parametrów, kolorowy output
REST API	FastAPI 0.100+	Asynchroniczność, automatyczna dokumentacja OpenAPI, walidacja Pydantic, wysoka wydajność
Excel	openpyxl 3.1+	Wsparcie dla xlsx, formatowanie warunkowe, wykresy, formuły
PDF	reportlab 4.0+	Niskopoziomowa kontrola nad layoutem, obsługa polskich znaków, wykresy
Wykresy	matplotlib 3.8+, seaborn 0.13+	Standardowe biblioteki wizualizacji, eksport PNG/SVG, integracja z reportlab
Testy	pytest 7.4+, pytest-cov	De facto standard testowania w Pythonie, fixtures, parametryzacja, pokrycie kodu
Analiza kodu	Black, isort, Flake8, MyPy	Automatyczne formatowanie, sortowanie importów, analiza statyczna, sprawdzanie typów
Konteneryzacja	Docker	Powtarzalność środowiska, izolacja zależności, łatwe wdrażanie
CI/CD	GitHub Actions	Integracja z repozytorium, darmowe minuty, bogaty marketplace actions
Konfiguracja	PyYAML	Czytelny format, hierarchiczna struktura, wsparcie dla komentarzy
Logging	loguru	Prostsza konfiguracja niż stdlib logging, kolorowy output, rotacja plików

Źródło: Opracowanie własne

3.4.2 Rozważane alternatywy

Rozważano Go (wydajność) – wybrano Python (boto3, raportowanie). Django REST Framework – wybrano FastAPI (asynchroniczność, OpenAPI). WeasyPrint/FPDF2 – wybrano reportlab (kontrola layoutu, brak zależności).

3.4.3 Zależności projektu

Plik `requirements.txt` zawiera 28 bezpośrednich zależności, pogrupowanych funkcjonalnie:

```
boto3>=1.34  click>=8.1  fastapi>=0.100  openpyxl>=3.1
reportlab>=4.0  matplotlib>=3.8  pytest>=8.0  # + 20 innych
```

Listing 3.6. Główne zależności projektu
Źródło: Opracowanie własne

3.5 Środowisko testowe

3.5.1 Konfiguracja środowiska testowego (mock)

Testy realizowano w trybie mock, wykorzystując dane symulowane przechowywane w repozytorium. Środowisko obejmowało trzy regiony: us-east-1 (Virginia), us-west-2 (Oregon), eu-west-1 (Ireland).

Zasoby (mock): 4 instancje EC2, 5 wolumenów EBS, 5 bucketów S3, 3 instancje RDS, 14 zasobów VPC, 14 zasobów IAM, 4 funkcje Lambda, 5 Security Groups, 35 rekordów Cost Explorer.

Łącznie środowisko testowe zawierało **89 zasobów AWS**.

3.5.2 Środowisko developerskie

Projekt jest przenośny na macOS/Ubuntu/Windows i uruchamiany w Python 3.11 w środowisku wirtualnym `venv` oraz Docker Compose.

3.5.3 Dane testowe (tryb mock)

Dla testów bez dostępu do AWS przygotowano 9 plików JSON w katalogu `data/mock/`, symulujących odpowiedzi API (łącznie 89 zasobów). Porównanie obu środowisk przedstawia tabela 3.3.

Zestaw mock umożliwia powtarzalne testy funkcjonalne bez dostępu do konta AWS oraz prezentację działania platformy w repozytorium. Wartość czasu skanowania ma charakter orientacyjny i nie pochodzi z formalnego benchmarku.

Tab. 3.3. Podsumowanie środowiska testowego

Aspekt	**Produkcja (poza zakresem testów)**	**Mock (repozytorium)**
Liczba zasobów	n/d	89
Regiony	n/d	3 (us-east-1, us-west-2, eu-west-1)
Wykryte niezgodności	n/d	73
Czas skanowania	n/d	~ 8 s (mock, pomiar orientacyjny)
Koszt	n/d	0
Dostępność	n/d	Zawsze dostępne
Użycie	n/d	Testy jednostkowe i demonstracyjne

Źródło: Opracowanie własne

ROZDZIAŁ 4

IMPLEMENTACJA I WYNIKI

4.1 Warstwa kolekcji danych (Collection Layer)

4.1.1 Abstrakcyjna klasa bazowa BaseCollector

Podstawą warstwy kolekcji jest abstrakcyjna klasa `BaseCollector`, definiująca interfejs dla kolektorów usług AWS. Klasa implementuje wzorzec Template Method [19]: metoda `collect()` definiuje algorytm kolekcji, a podklasy implementują kroki specyficzne dla usługi.

```
class BaseCollector(ABC):
    @abstractmethod
    def _get_service_name(self) -> str: pass
    @abstractmethod
    def _collect_resources(self, client, region) -> List[Dict]:
    pass

    def collect(self, regions: List[str]) -> Dict[str, Any]:
        for region in regions:
            client = self.session.get_client(self.service_name,
    region)
            resources = self._collect_resources(client, region)
        return {"resources": all_resources, "by_region": by_region,
                "metadata": {...}}
```

Listing 4.1. Implementacja klasy BaseCollector

Źródło: Opracowanie własne na podstawie `src/collectors/base_collector.py`

Klasa implementuje wzorzec Template Method z obsługą błędów per-region (błąd w jednym regionie nie przerywa kolekcji), metadanymi kolekcji i logowaniem ustrukturyzowanym (loguru).

4.1.2 Implementacja kolektorów usług

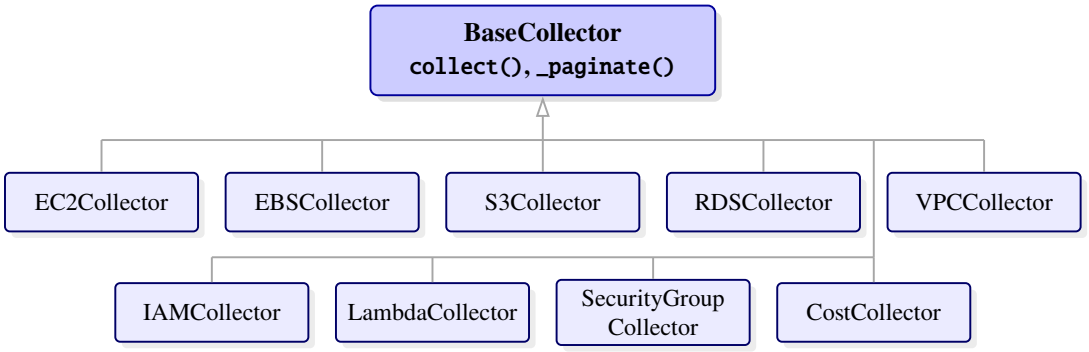
Każda z dziewięciu usług AWS posiada dedykowany kolektor dziedziczący po klasie `BaseCollector`. Każdy kolektor wykorzystuje paginatory boto3 i transformuje surowe dane do formatu standardowego. Tabela 4.1 przedstawia charakterystykę wszystkich kolektorów.

Tab. 4.1. Charakterystyka kolektorów usług AWS

Kolektor	**API AWS**	**Zbierane atrybuty**	**LOC**
EC2Collector	describe_instances	ID, typ, stan, VPC, SG, monitoring	248
EBSCollector	describe_volumes	ID, rozmiar, typ, szyfrowanie, snapshoty	242
S3Collector	list_buckets + get_*	nazwa, wersjonowanie, szyfrowanie, ACL	385
RDSCollector	describe_db_instances	ID, engine, szyfrowanie, backup, public	312
VPCCollector	describe_vpcs + subnets	VPC ID, CIDR, subnety, IGW, flow logs	308
IAMCollector	list_users/roles/policies	użytkownicy, role, polityki, MFA, klucze	359
LambdaCollector	list_functions	nazwa, runtime, VPC, env vars, memory	224
SecurityGroupCollector	describe_security_groups	ID, reguły ingress/egress, przypisania	356
CostCollector	get_cost_and_usage	koszt per usługa, trend, nieużywane	255
		Razem:	**2689**

Źródło: Opracowanie własne

Diagram klas warstwy kolekcji przedstawiono na rysunku 4.1.



Rys. 4.1. Diagram klas warstwy kolekcji

Źródło: Opracowanie własne

4.1.3 Obsługa paginacji API AWS

Wiele API AWS limituje liczbę wyników (typowo 100–1000 elementów). Problem rozwiązano paginatorami boto3 i własną implementacją generatora dla API bez wsparcia paginatorów.

Paginatory boto3 automatycznie obsługują tokeny kontynuacji, eliminując potrzebę ręcznego zarządzania stanem. Dla usług bez wsparcia paginatorów zaimplementowano generator `_paginate_results()` z automatycznym wykrywaniem końca danych.

4.2 Warstwa analizy zgodności (Analysis Layer)

4.2.1 ComplianceAnalyzer – rdzeń systemu

Moduł `ComplianceAnalyzer` (2535 linii) to główny komponent platformy, zawierający 58 reguł bezpieczeństwa. Architektura oparta jest na wzorcu Chain of Responsibility – każda reguła jest niezależnym obiektem sprawdzającym aspekt konfiguracji.

Każda reguła jest obiektem `ComplianceRule` zawierającym: identyfikator, krytyczność, funkcję sprawdzającą, instrukcję naprawy oraz mapowanie na frameworki zgodności
(SOC 2, HIPAA, PCI-DSS).

4.2.2 Algorytm oceny zgodności

Wskaźnik zgodności obliczany jest według wzoru uwzględniającego wagę krytyczności: $\text{Score} = 100 \times \left(1 - \frac{\sum w_i}{\sum w_j}\right)$, gdzie $\sum w_i$ oznacza sumę wag naruszeń wykrytych w danym skanowaniu (każde naruszenie wnosi wagę odpowiadającą jego krytyczności), a $\sum w_j$ – sumę wag wszystkich wykonanych sprawdzeń, tj. par (reguła, zasób), niezależnie od wyniku. Wagi krytyczności: Critical=10, High=5, Medium=2, Low=1, Info=0. Przykładowo, jeśli platforma sprawdzi 58 reguł na 89 zasobach, to $\sum w_j$ obejmuje wagi dla każdego takiego sprawdzenia (zakładając jednorodną krytyczność reguły niezależnie od zasobu).

4.2.3 Moduły frameworków zgodności

Mapowanie na frameworki SOC 2, HIPAA i PCI-DSS zaimplementowano jako osobne moduły w katalogu `src/analyzers/frameworks/`:

- `soc2.py` (561 linii) – mapowanie na 5 Trust Service Criteria
- `hipaa.py` (523 linie) – mapowanie na 3 kategorie Safeguards
- `pci_dss.py` (653 linie) – mapowanie na 12 Requirements

Identyfikatory CIS w tabeli poniżej odzwierciedlają mapowanie zastosowane w implementacji narzędzia.

Tab. 4.2. Fragment mapowania reguł CIS na frameworki zgodności

CIS ID	Kontrola	SOC 2	HIPAA	PCI-DSS
1.4	Brak kluczy dostępu root	CC6.1, CC6.2	164.312(d)	8.1.1
1.5	MFA dla konta root	CC6.1	164.312(d)	8.3.1
2.1.1	S3: blokada żądań HTTP	CC6.1, CC6.7	164.312(e)(2)	4.1
2.1.2	S3: szyfrowanie po stronie serwera	CC6.1	164.312(a)(1)	3.4
2.2.1	EBS: szyfrowanie włączone	CC6.1	164.312(a)(1)	3.4.1
3.1	CloudTrail włączony	CC7.2, CC7.3	164.312(b)	10.1
4.1	SG: brak reguły 0.0.0.0/0 dla SSH	CC6.1, CC6.6	164.312(e)(1)	1.3.1

Źródło: Opracowanie własne

4.3 Warstwa raportowania (Reporting Layer)

4.3.1 ExcelReporter – raport korporacyjny

`ExcelReporter` generuje wieloarkuszowy plik xlsx dla zespołów compliance i audytu. Raport ma jedenaście arkuszy:

1. **Podsumowanie wykonawcze** (Executive Summary)
2. **Inwentarz EC2** (EC2 Inventory)
3. **Inwentarz EBS** (EBS Inventory)
4. **Zasobniki S3** (S3 Buckets)
5. **Bazy danych RDS** (RDS Databases)
6. **Grupy zabezpieczeń** (Security Groups)
7. **Niezgodności** (Compliance Issues)
8. **VPC** (VPC Inventory)
9. **IAM** (IAM Inventory)
10. **Lambda** (Lambda Inventory)
11. **Koszty** (Costs)

Formatowanie warunkowe automatycznie koloruje komórki według krytyczności: Critical (czerwone), High (pomarańczowe), Medium (żółte), Low (zielone).

```python
class ExcelReporter:
    SEVERITY_COLORS = {"CRITICAL": "FF6B6B", "HIGH": "FFB347", ...}

    def generate(self, scan_result: Dict, output_path: str):
        wb = Workbook()
        self._write_summary(wb.active, scan_result)
        for service in ["ec2", "ebs", "s3", "rds", "sg"]:
            self._write_inventory_sheet(wb, service, scan_result)
        self._write_issues(wb.create_sheet("Niezgodnosci"),
    scan_result)
        self._apply_conditional_formatting(ws_issues)  # kolory wg
    severity
        wb.save(output_path)
```

Listing 4.2. Struktura ExcelReporter (fragment, uproszczony)

Źródło: Opracowanie własne na podstawie `src/reporters/excel_reporter.py`

Tab. 4.3. Podsumowanie wykonawcze raportu Excel – wyniki skanowania testowego

Metryka	**Wartość**	**Status**	**Szczegóły**
Wskaźnik zgodności	68%	Średni	58 reguł sprawdzonych
Zasoby przeskanowane	89	–	3 regiony AWS
Znaleziska krytyczne	6	Krytyczny	–
Znaleziska wysokie	12	Wysoki	–
Znaleziska średnie	37	Średni	–
Znaleziska niskie	14	Niski	–
Znaleziska informacyjne	4	Informacyjny	–
Czas skanowania	~8 s	–	Tryb mock (dane symulowane; 3 regiony, 89 zasobów, raport Excel)

Źródło: Opracowanie własne – dane z raportu mock

Przykładowe podsumowanie wykonawcze raportu Excel przedstawia tabela 4.3. Pełną listę jedenastu arkuszy podano na stronie 37.

4.3.2 ChartGenerator – wizualizacje

Moduł `ChartGenerator` (1344 linie) wykorzystuje biblioteki matplotlib i seaborn do generowania wykresów. Wykresy są eksportowane jako pliki PNG (dla raportów PDF/HTML) oraz osadzane bezpośrednio w arkuszach Excel.

Typy generowanych wykresów: kołowy (pie chart), słupkowy (bar chart), słupkowy skumulowany (stacked bar), mapa ciepła (heatmap) oraz linia trendu (trend line).

4.4 REST API

4.4.1 Architektura API

REST API oparte na FastAPI umożliwia integrację z systemami zewnętrznymi (panele, SIEM, systemy zgłoszeń). API jest asynchroniczne – skanowanie uruchamia się w tle, a klient odpytuje endpoint statusu.

```python
app = FastAPI(title="AWS Infrastructure Intelligence API", version="1.0.0")

class ScanRequest(BaseModel):
    regions: List[str] = ["us-east-1"]
    services: List[str] = ["ec2", "s3", "iam"]
    mock_mode: bool = False

@app.post("/api/v1/scans")
async def create_scan(request: ScanRequest, background_tasks: BackgroundTasks):
    scan_id = str(uuid.uuid4())
    background_tasks.add_task(run_scan_async, scan_id, request)
    return {"scan_id": scan_id, "status": "pending"}

@app.get("/api/v1/scans/{scan_id}")
async def get_scan_status(scan_id: str): return scans.get(scan_id)
```

Listing 4.3. Endpointy REST API

Źródło: Opracowanie własne na podstawie `src/api/main.py`

4.4.2 Dokumentacja OpenAPI

FastAPI automatycznie generuje dokumentację OpenAPI (Swagger) dostępną pod `/docs`. Dokumentacja zawiera schematy request/response, możliwość testowania endpointów oraz przykłady użycia.

4.5 Pipeline CI/CD

4.5.1 Struktura przepływu pracy w GitHub Actions

Pipeline CI/CD składa się z sześciu etapów uruchamianych przy każdym push do gałęzi main/develop oraz przy pull requestach. Workflow zdefiniowany jest w pliku YAML w katalogu `.github/workflows/`.

```yaml
name: CI/CD Pipeline
on: [push, pull_request]
jobs:
    lint:        # Black, isort, Flake8, MyPy
    test:        # pytest na Python 3.9-3.12, pokrycie Codecov
    integration: # testy integracyjne w trybie mock
    security:    # Bandit (SAST), Safety (zależności)
    build:       # Docker build + test w trybie mock
    docs:        # walidacja dokumentacji
```

Listing 4.4. Struktura przepływu pracy CI/CD

Źródło: Opracowanie własne na podstawie `.github/workflows/ci.yml`

4.5.2 Etapy pipeline'u

Tabela 4.4 przedstawia wszystkie etapy pipeline'u z czasami wykonania.

Tab. 4.4. Etapy pipeline'u CI/CD z czasami wykonania

Etap	**Opis**	**Czas (szac.)**	**Narzędzia**
Analiza kodu	Analiza statyczna kodu	45s	Black, isort, Flake8, MyPy
Testy	Testy jednostkowe (4 wersje Python)	3m	pytest, pytest-cov
Integracja	Testy integracyjne (tryb mock)	90s	pytest markers
Bezpieczeństwo	Skanowanie bezpieczeństwa	60s	Bandit, Safety
Budowanie	Budowanie obrazu Docker	2m	Docker wieloetapowy
Dokumentacja	Walidacja dokumentacji	30s	markdown-lint
Całkowity czas (szacowany):		**ok. 9 min**	

Źródło: Opracowanie własne

Czasy mają charakter orientacyjny i zależą od obciążenia runnerów oraz konfiguracji środowiska.

4.6 Wyniki testów

4.6.1 Testy automatyczne

Pakiet testów obejmuje łącznie 87 testów w trzech plikach testowych i jest uruchamiany w pipeline'ie CI. Testy oznaczone są markerami pytest: 80 testów jednostkowych (`@pytest.mark.unit`) oraz 7 testów integracyjnych (`@pytest.mark.integration`).

Struktura testów w rozbiciu na pliki:

- `tests/test_collectors.py` – 27 testów kolektorów (25 jednostkowych, 2 integracyjne)
- `tests/test_analyzers.py` – 32 testy analizatorów (30 jednostkowych, 2 integracyjne)
- `tests/test_reporters.py` – 28 testów reporterów (25 jednostkowych, 3 integracyjne)

Poniżej przedstawiono reprezentatywne przykłady testów jednostkowych. Listing 4.5 ilustruje test kolektora EC2, który weryfikuje poprawność parsowania odpowiedzi API AWS w trybie mock:

```
@pytest.mark.unit
@pytest.mark.collectors
def test_collect_returns_correct_structure(mock_session_manager):
    from src.collectors.ec2_collector import EC2Collector

    collector = EC2Collector(mock_session_manager, mock_mode=True)
    result = collector.collect(["us-east-1"])

    assert "resources" in result
    assert "by_region" in result
    assert "metadata" in result
```

Listing 4.5. Test jednostkowy kolektora EC2

Źródło: Opracowanie własne na podstawie `tests/test_collectors.py`

Listing 4.6 przedstawia test reguły CIS weryfikującej szyfrowanie wolumenów EBS:

```python
@pytest.mark.unit
@pytest.mark.analyzers
def test_detects_unencrypted_ebs(sample_ebs_collector_result):
    from src.analyzers.compliance_analyzer import
    ComplianceAnalyzer

    resources = {"ebs": sample_ebs_collector_result}
    analyzer = ComplianceAnalyzer()
    result = analyzer.analyze(resources)

    ebs_issues = [
        i for i in result["issues"]
        if "EBS" in i["rule_id"] and "encrypt" in i["finding"].
    lower()
    ]

    assert len(ebs_issues) >= 1
```

Listing 4.6. Test reguły CIS-2.2.1 (szyfrowanie EBS)

Źródło: Opracowanie własne na podstawie `tests/test_analyzers.py`

4.6.2 Wyniki skanowania środowiska testowego

Skanowanie środowiska testowego (89 zasobów, 3 regiony) wykazało rezultaty przedstawione w tabeli 4.5.

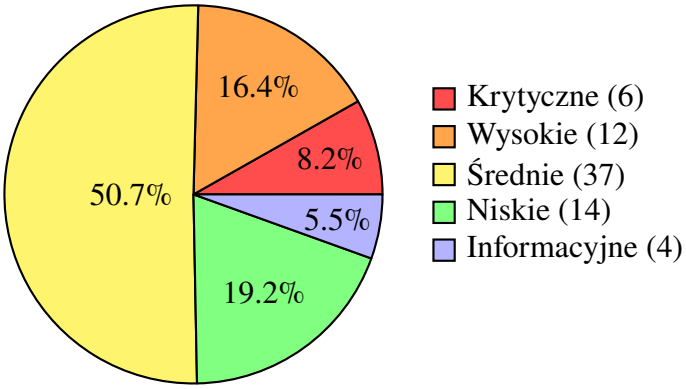
Tab. 4.5. Wyniki skanowania środowiska testowego

Metryka	**Opis**	**Wartość**
Zasoby przeskanowane	Łączna liczba zasobów AWS	89
Regiony	Liczba regionów	3
Reguły sprawdzone	Liczba reguł w analizatorze	58
Znaleziska	Łączna liczba niezgodności	73
– Critical	Znaleziska krytyczne	6
– High	Znaleziska wysokie	12
– Medium	Znaleziska średnie	37
– Low	Znaleziska niskie	14
– Info	Informacyjne	4
Wskaźnik zgodności	Wartość wskaźnika	68%
Czas pełnego cyklu	Kolekcja + analiza + raportowanie (mock, pomiar orientacyjny; 3 regiony, 89 zasobów, raport Excel)	~ 8 s
Pamięć szczytowa	Szczytowe zużycie RAM	ok. 150 MB

Źródło: Opracowanie własne

Wymaganie NF1 (500 zasobów <5 min) nie zostało zweryfikowane empirycznie; pomiary obejmują jedynie zestaw mock 89 zasobów.

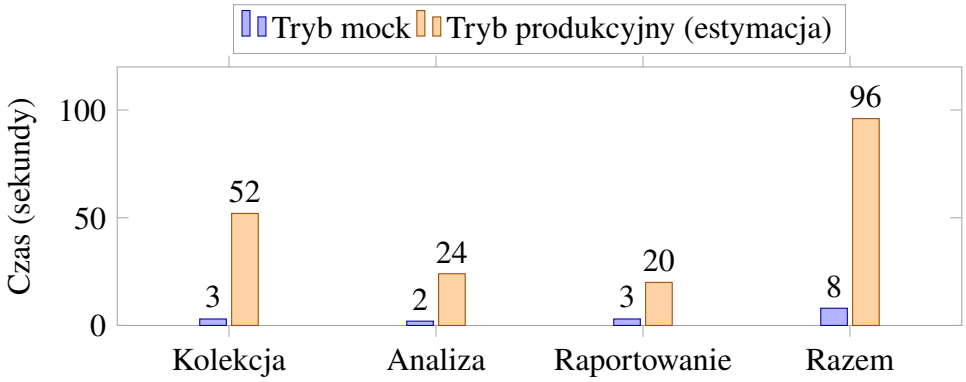
Rozkład znalezisk według krytyczności przedstawia rysunek 4.2.



Rys. 4.2. Rozkład znalezisk według poziomu krytyczności

Źródło: Opracowanie własne

Porównanie czasu skanowania w trybie mock i szacunkowego czasu w trybie produkcyjnym przedstawia rysunek 4.3. **Uwaga:** wartości dla trybu produkcyjnego są estymacją opartą na typowej latencji API AWS, nie na pomiarach. Nie przeprowadzono benchmarku na rzeczywistym środowisku, dlatego dane produkcyjne na wykresie mają charakter wyłącznie orientacyjny.



Rys. 4.3. Porównanie czasu skanowania: tryb mock vs produkcyjny (estymacja)

Źródło: Opracowanie własne

4.6.3 Wyniki skanowania w rozbiciu na usługi

Obecna wersja raportu JSON z trybu mock nie zawiera szczegółowych metryk per usługa (np. wskaźnika zgodności per usługa). Rozszerzenie raportowania o taką granulację jest jednym z kierunków dalszego rozwoju platformy.

4.6.4 Uwagi dotyczące walidacji

W repozytorium udostępniono raport z trybu mock, który nie zawiera kompletnej analizy trafności (precision/recall) ani porównań z narzędziami zewnętrznymi. Rozszerzenie walidacji o pełne metryki i benchmarki wymaga oddzielnego środowiska testowego oraz zapisanych wyników referencyjnych.

4.7 Wkład własny autora

Niniejsza sekcja omawia szczegóły implementacyjne wkładu własnego zdefiniowanego w sekcji 1.7.

4.7.1 Zunifikowany mapper wieloframeworkowy

Mapper pozwala zdefiniować regułę raz z automatycznym mapowaniem na wszystkie frameworki. W analizowanych wersjach Prowlera i ScoutSuite mapowanie frameworków wymaga osobnej konfiguracji lub implementacji, co może prowadzić do duplikacji kodu i rozbieżności w wynikach.

Opracowany mapper wykorzystuje dekoratory Python do rejestracji reguł:

```
1 @compliance_rule("CIS-1.4", {"soc2": ["CC6.1"], "hipaa": ["164.312(
      d)"]})
2 def check_no_root_keys(resources): ...  # automatyczna rejestracja
```

Listing 4.7. Dekorator mapowania reguły
Źródło: Opracowanie własne

4.7.2 Wzorzec architektury dwutrybowej

Wzorzec architektury dwutrybowej (sekcja 3.2.4) ułatwia testowanie systemów zależnych od zewnętrznych API i może być przydatny w projektach wymagających symulacji API chmurowych.

4.7.3 Pakiet raportowania

Reportery generują formaty, których Prowler i ScoutSuite nie oferują natywnie: wieloarkuszowy Excel z formatowaniem warunkowym, PDF z wykresami oraz interaktywny HTML z osadzonymi danymi.

4.8 Napotkane wyzwania techniczne

4.8.1 Obsługa paginacji boto3

Problem może się ujawnić w środowiskach z dużą liczbą zasobów. Metoda boto3 `list_users()` domyślnie zwraca maksymalnie 100 wyników, co bez paginacji prowadzi do niekompletnych danych (np. przy 947 użytkownikach kolektor pobierze jedynie pierwszych 100).

Rozwiązanie wymagało dwuetapowej refaktoryzacji:

1. Zamiana bezpośrednich wywołań API na paginatory boto3 (`get_paginator()`)
2. Implementacja generatora `_paginate_results()` dla API bez wsparcia paginatorów

Czas implementacji: 1,5 dnia. Liczba zmienionych linii: 234.

4.8.2 Ograniczanie przepustowości API AWS

W trybie produkcyjnym równoległe skanowanie wielu regionów może powodować błędy typu
`ThrottlingException` – AWS limituje liczbę żądań API na sekundę. Problem nasila się przy dużych środowiskach, dlatego zaimplementowano mechanizm prewencyjny.

Implementacja rozwiązania obejmowała:

- Wykładnicze opóźnienie z losowością (0–500ms)
- Maksymalnie 5 prób przed zgłoszeniem błędu
- Ograniczenie współbieżności do 5 wątków per region

```
def retry_with_backoff(func, max_retries=5):
    for attempt in range(max_retries):
        try: return func()
        except ThrottlingException:
            time.sleep((2 ** attempt) + random.uniform(0, 0.5))
```

Listing 4.8. Wykładnicze opóźnienie
Źródło: Opracowanie własne

4.8.3 Obsługa polskich znaków w raportach PDF

Reportlab domyślnie nie obsługuje znaków spoza ASCII. Nazwy zasobów AWS z polskimi znakami (np. „Serwer-główny”) były renderowane jako kwadraty lub powodowały błędy.

Rozwiązanie wymagało:

1. Rejestracji czcionek TTF z pełnym zestawem Unicode (DejaVu Sans)
2. Konfiguracji reportlab do używania zarejestrowanych czcionek
3. Fallback na czcionkę domyślną dla znaków spoza zakresu

```
pdfmetrics.registerFont(TTFont('DejaVu', 'fonts/DejaVuSans.ttf'))
styles['Normal'].fontName = 'DejaVu'  # pełna obsługa polskich znak
    ów
```

Listing 4.9. Czcionki Unicode

Źródło: Opracowanie własne

4.8.4 Podsumowanie wyzwań

Tabela 4.6 podsumowuje napotkane wyzwania z czasem potrzebnym na rozwiązanie.

Tab. 4.6. Napotkane wyzwania techniczne i ich rozwiązania

Wyzwanie	Rozwiązanie	Pliki	Czas
Paginacja API	Generatory + paginatory boto3	collectors/*.py	1,5 dnia
Ograniczanie przepustowości	Wykładnicze opóźnienie z losowością	utils/aws_session.py	1 dzień
Unicode w PDF	Czcionki DejaVu TTF	`reporters/pdf_reporter.py`	0,5 dnia
Tryb mock	Wzorzec Factory + MockClient	`utils/aws_session.py`	2 dni
Równoległość	`ThreadPoolExecutor` z limitem	`main.py`	1 dzień

Źródło: Opracowanie własne

ROZDZIAŁ 5

ZAKOŃCZENIE

5.1 Podsumowanie pracy

Praca opisuje projekt i implementację platformy AWS Infrastructure Intelligence – systemu CSPM do automatycznej walidacji zgodności infrastruktury AWS ze standardami bezpieczeństwa.

Wszystkie cele z rozdziału pierwszego zostały zrealizowane w zakresie pracy; walidacja ma charakter wstępny i opiera się na trybie mock.

Analiza porównawcza narzędzi CSPM (**cel poznawczy**) objęła rozwiązania komercyjne (Prisma Cloud, Wiz, AWS Security Hub) oraz open-source (Prowler, ScoutSuite, Cloud Custodian). Żadne z analizowanych darmowych narzędzi nie łączy wieloframeworkowej walidacji z natywnym raportowaniem w formatach korporacyjnych (Excel, PDF) – szczegóły w sekcji 2.1.4.

Na tej podstawie opracowano czterowarstwową architekturę (**cel projektowy**): warstwa wejściowa, kolekcji, analizy i raportowania, z wzorcami Template Method (kolektory), Chain of Responsibility (reguły) oraz Factory (tryb dwutrybowy). Architektura ułatwia rozszerzanie systemu o nowe moduły.

Powstała platforma (**cel implementacyjny**) liczy około 17 659 linii kodu Pythona w 43 plikach (w tym testy) i obejmuje:

- 9 kolektorów usług AWS (EC2, EBS, S3, RDS, VPC, IAM, Lambda, Security Groups, Cost Explorer),
- 58 reguł bezpieczeństwa z mapowaniem na 4 frameworki (CIS, SOC 2, HIPAA, PCI-DSS),
- 5 formatów raportów (Excel z 11 arkuszami, PDF, HTML, JSON, wykresy),
- REST API (FastAPI) z dokumentacją OpenAPI,
- CLI (Click) z pełnym zestawem komend,
- tryb mock umożliwiający testy bez dostępu do AWS,
- 6-etapowy pipeline CI/CD w GitHub Actions,
- konteneryzację Docker z wieloetapowym budowaniem.

Testy w trybie mock na zestawie 89 zasobów w 3 regionach (**cel walidacyjny**) wykazały 73 niezgodności przy wskaźniku zgodności 68%.

Tabela 5.1 zestawia cele pracy z osiągniętymi rezultatami.

Tab. 5.1. Weryfikacja realizacji celów pracy

Cel	Opis realizacji	Status
Poznawczy	Analiza porównawcza narzędzi CSPM, identyfikacja luk	Zrealizowany
Projektowy	Architektura 4-warstwowa, 3 wzorce projektowe	Zrealizowany
Implementacyjny	ok. 17 659 LOC, 9 kolektorów, 58 reguł, 5 formatów	Zrealizowany
Walidacyjny	Testy mock: 89 zasobów, 73 niezgodności, wskaźnik 68% (bez weryfikacji NF1)	Częściowo zrealizowany

Źródło: Opracowanie własne

5.2 Weryfikacja hipotez badawczych

Trzy hipotezy badawcze sformułowane w sekcji 1.4 zweryfikowano następująco: H1 i H2 empirycznie, H3 w sposób wstępny (ocena jakościowa).

H1: *Można zaprojektować system CSPM walidujący zgodność z wieloma frameworkami bez duplikacji logiki.*

Hipoteza **potwierdzona**. Zunifikowany mapper wieloframeworkowy pozwala zdefiniować regułę raz z automatycznym mapowaniem na CIS, SOC 2, HIPAA i PCI-DSS. Mechanizm dekoratorów umożliwia mapowanie jednej reguły na wiele frameworków bez duplikacji logiki walidacyjnej. Weryfikacja: 58 reguł mapowanych na 4 frameworki przez jeden mechanizm dekoratorów, co eliminuje potrzebę osobnej implementacji logiki sprawdzającej dla każdego frameworka.

H2: *Architektura dwutrybowa umożliwia integrację CSPM z CI/CD bez dostępu do produkcji.*

Hipoteza **potwierdzona**. Tryb mock z MockAWSClient pozwala uruchomić skanowanie z danymi symulowanymi. Pipeline CI/CD w GitHub Actions wykonuje testy integracyjne bez credentials AWS, co potwierdza możliwość integracji CSPM z procesem CI/CD bez dostępu do produkcji.

H3: *Raportowanie w formatach korporacyjnych (Excel) zwiększa użyteczność dla zespołów compliance.*

Hipoteza **wstępnie potwierdzona** na podstawie oceny jakościowej. Analiza wygenerowanych raportów Excel wskazuje na przydatność następujących elementów:

- arkusz *Executive Summary* – pozwala na szybką ocenę stanu zgodności bez analizy szczegółów technicznych,

- formatowanie warunkowe kolorami – intuicyjna wizualizacja poziomu krytyczności,
- arkusz *Compliance Issues* z kolumną *Remediation* – bezpośrednio użyteczny dla zespołów operacyjnych.

Pełna walidacja statystyczna wymaga badania na większej próbie z zastosowaniem standaryzowanych kwestionariuszy (np. SUS/TAM). W ramach niniejszej pracy przedstawiono wstępną ocenę jakościową, która potwierdza kierunek przyjętych rozwiązań.

5.3 Wkład inżynierski i praktyczny

5.3.1 Wkład inżynierski

Główne wyniki inżynierskie pracy (szczegóły implementacyjne w sekcji 4.7):

1. **Zunifikowany mapper wieloframeworkowy.** Każda reguła bezpieczeństwa jest definiowana raz i za pomocą dekoratorów automatycznie mapowana na wszystkie frameworki (CIS, SOC 2, HIPAA, PCI-DSS). Nie trzeba duplikować logiki walidacyjnej, a dodanie nowego frameworka sprowadza się do rozszerzenia słownika mapowań.
2. **Wzorzec architektury dwutrybowej.** Pozwala przełączać się między trybem produkcyjnym (API AWS) a testowym (dane symulowane) bez modyfikacji logiki biznesowej. Wzorzec można zastosować w dowolnych systemach zależnych od zewnętrznych API chmurowych i ułatwia integrację z pipeline'ami CI/CD.
3. **Algorytm ważonej oceny zgodności.** Wskaźnik zgodności obliczany jest z wagami krytyczności (Critical=10, High=5, Medium=2, Low=1), co daje dokładniejszy obraz stanu bezpieczeństwa niż proste zliczanie niezgodności – duża liczba drobnych problemów nie przesłania nielicznych, ale krytycznych naruszeń.

5.3.2 Wkład praktyczny

Z perspektywy praktycznej platforma AWS Infrastructure Intelligence:

1. Jest darmową alternatywą dla komercyjnych narzędzi CSPM (Prisma Cloud, Wiz, AWS Security Hub), udostępnioną na licencji MIT.
2. Generuje raporty w formatach Excel i PDF, które są praktyczne dla zespołów audytu i compliance, co wynika ze wstępnej oceny jakościowej opisanej w sekcji 5.2.
3. Integruje się z procesami DevSecOps przez REST API i wbudowany tryb CI/CD – nie wymaga konfigurowania zewnętrznych narzędzi mockowania.

4. Dzięki modularnej architekturze kod źródłowy można rozszerzać o nowe usługi AWS, frameworki zgodności i formaty raportów.

5.4 Ograniczenia pracy

Platforma ma następujące ograniczenia, które należy uwzględnić przy jej wdrażaniu:

Ograniczenie do Amazon Web Services. Obecna wersja obsługuje wyłącznie AWS. Część organizacji utrzymuje zasoby w wielu chmurach, a narzędzia CSPM oferują wsparcie multi-cloud (np. Microsoft Defender for Cloud obejmuje AWS i GCP) [24]. Architektura abstrahuje interakcję z chmurą przez `BaseCollector`, co ułatwia przyszłe rozszerzenie, jednak implementacja kolektorów dla Azure (azure-sdk-for-python) i GCP (google-cloud-python) wymaga napisania nowych modułów od podstaw, z uwzględnieniem odmiennych modeli uprawnień i API.

Brak automatycznej remediacji. Platforma identyfikuje niezgodności i dostarcza instrukcje naprawy w polu `remediation`, lecz nie wykonuje zmian w infrastrukturze automatycznie. Decyzja ta była świadoma – automatyczna remediacja w środowisku produkcyjnym niesie ryzyko: wyłączenie publicznego dostępu do bucketa S3 może zablokować działającą aplikację, a wymuszenie szyfrowania EBS wymaga restartu instancji. Bezpieczna implementacja wymaga mechanizmu zatwierdzania zmian (*approval workflow*) i rollbacku, co wykracza poza zakres niniejszej pracy.

Ograniczona obsługa usług AWS. Platforma obsługuje 9 najpopularniejszych usług. Usługi takie jak EKS (Kubernetes), ECS (kontenery), DynamoDB (baza NoSQL), Kinesis (streaming) czy Step Functions (orkiestracja) nie są obecnie skanowane. W zestawie mock 9 usług stanowi większość zasobów, jednak pełne pokrycie wymaga dalszego rozwoju.

Brak interfejsu webowego. Interakcja z platformą odbywa się poprzez CLI lub REST API. Dla użytkowników nietechnicznych może to stanowić barierę wejścia. REST API umożliwia budowę panelu webowego jako osobnego komponentu, jednak sam panel nie został zaimplementowany w ramach niniejszej pracy.

Brak persystencji historycznej. Wyniki skanowań nie są przechowywane w bazie danych, co uniemożliwia analizę trendów zgodności w czasie i porównywanie stanów z różnych punktów czasowych. Każde skanowanie generuje niezależne raporty. Dodanie warstwy persystencji (np. PostgreSQL z modelem `ScanRun -> Findings`) jest priorytetem rozwojowym.

Niezweryfikowane wymaganie NF1. Wymaganie wydajnościowe NF1 (skanowanie 500 zasobów w czasie poniżej 5 minut) nie zostało zweryfikowane empirycznie; pomiary objęły jedynie zestaw mock (89 zasobów, ok. 8 s).

Wstępna walidacja H3. Hipoteza H3 dotycząca użyteczności raportów Excel została oceniona w sposób jakościowy.

Pełna walidacja wymaga badania na większej próbie użytkowników reprezentujących różne role (audytorzy, specjaliści compliance, inżynierowie DevOps), z zastosowaniem standaryzowanych kwestionariuszy, takich jak System Usability Scale (SUS) lub Technology Acceptance Model (TAM). W ramach niniejszej pracy przeprowadzono walidację wstępną, która potwierdza kierunek przyjętych rozwiązań projektowych, lecz nie stanowi dowodu statystycznego.

5.5 Kierunki dalszego rozwoju

Implementacja i testowanie platformy pozwoliły wskazać kierunki dalszego rozwoju:

- **Rozszerzenie pokrycia usług AWS** – dodanie kolektorów dla EKS, ECS, DynamoDB, CloudFront i Route 53. Wymaga implementacji nowych klas dziedziczących po `BaseCollector` oraz rozszerzenia `ComplianceAnalyzer` o odpowiednie reguły CIS (sekcje 4.x i 5.x benchmarku).
- **Mechanizm wyjątków** (*suppressions*) – możliwość oznaczenia znanych fałszywych alarmów jako zaakceptowanych, z uzasadnieniem i datą wygaśnięcia, co usunie powtarzalne false positives z raportów (sekcja 4.6.4).
- **Panel webowy** – interfejs komunikujący się z istniejącym REST API, prezentujący wyniki skanowania w formie dashboardu.
- **Wsparcie Azure i GCP** – implementacja kolektorów dla Azure (Resource Manager, Key Vault, NSG) i GCP (Compute Engine, Cloud Storage, IAM). Wymaga abstrakcji wspólnego interfejsu `CloudProvider` ponad istniejącą klasą `AWSSessionManager`.
- **Automatyczna remediacja** – moduł wykonujący zatwierdzone naprawy z mechanizmem rollbacku. Każda remediacja generowałaby plan zmian (*dry-run*), wymagający zatwierdzenia przed wykonaniem.
- **Persystencja i trendy** – warstwa bazy danych przechowująca historię skanowań, co pozwoli na analizę trendów zgodności i generowanie raportów porównawczych.
- **Integracja SIEM/SOAR** – eksport znalezisk do systemów typu Splunk czy ELK w formacie CEF/LEEF.
- **Wykrywanie anomalii za pomocą uczenia maszynowego** – identyfikacja nietypowych wzorców konfiguracji wykraczających poza statyczne reguły CIS. Kierunek ten wymaga zebrania danych treningowych z rzeczywistych środowisk.
- **Certyfikacja CIS SecureSuite** – uzyskanie oficjalnej certyfikacji CIS potwierdzającej zgodność narzędzia z benchmarkiem.

5.6 Wnioski końcowe

Wnioski z przeprowadzonych prac:

1. **Automatyzacja walidacji zgodności jest praktycznie uzasadniona.** Manualne audyty okresowe nie nadążają za tempem zmian w środowiskach chmurowych. Już w testowanym zestawie mock (89 zasobów, 3 regiony) platforma wykryła 73 niezgodności w ok. 8 sekund. W środowisku produkcyjnym, gdzie liczba zasobów jest o rząd wielkości większa, ręczna weryfikacja każdej konfiguracji staje się niepraktyczna.

2. **Istnieje luka między narzędziami komercyjnymi a open-source w zakresie raportowania.** Rozwiązania komercyjne (Prisma Cloud, Wiz) są niedostępne dla organizacji z ograniczonym budżetem. Prowler i ScoutSuite wypełniają część tej luki, ale ich formaty wyjściowe (CSV, JSON, HTML) wymagają dodatkowej obróbki, zanim trafią do zespołu audytu. Opracowana platforma łączy zunifikowany mapper wieloframeworkowy z raportami w 5 formatach i trybem mock dla CI/CD.

3. **Wzorzec architektury dwutrybowej ma zastosowania poza CSPM.** Podział na tryb produkcyjny i mock (sekcja 3.2.4) sprawdził się nie tylko przy testowaniu reguł bezpieczeństwa, ale też przy demonstracji platformy bez konfiguracji konta AWS. Analogiczny wzorzec można zastosować w projektach zależnych od zewnętrznych API, np. przy testowaniu integracji z bramkami płatniczymi.

4. **Wybór Pythona okazał się trafny, choć nie bez kompromisów.** Oficjalny SDK (boto3) i biblioteki raportowania (openpyxl, reportlab, matplotlib) pozwoliły uniknąć pisania niskopoziomowego kodu integracyjnego. Wydajność Pythona jest niższa niż Go czy Rusta, co przy skanowaniu setek zasobów w trybie produkcyjnym może wymagać optymalizacji, jednak dla okresowych skanowań jest wystarczająca.

5. **Format raportu ma znaczenie dla odbiorcy.** Wstępna ocena jakościowa wskazuje, że arkusz *Executive Summary* z jednoekranowym podsumowaniem i formatowanie warunkowe (kolory wg krytyczności) ułatwiają pracę osobom nietechnicznym. Raporty HTML i JSON, generowane przez Prowlera i ScoutSuite, są użyteczne dla inżynierów, ale wymagają dodatkowego przetworzenia, zanim trafią do audytora.

Platforma AWS Infrastructure Intelligence jest przeznaczona zarówno dla zespołów DevOps (szybka walidacja, integracja z CI/CD), jak i działów compliance (raporty Excel i PDF). Kod źródłowy udostępniono na licencji MIT.

BIBLIOGRAFIA

[1] Mazhar Ali, Samee U. Khan i Athanasios V. Vasilakos. „Security in cloud computing: Opportunities and challenges”. W: *Information Sciences* 305 (2015), s. 357–383. DOI: 10.1016/j.ins.2015.01.025.

[2] Amazon Web Services. *AWS Well-Architected Framework - Security Pillar*. Dostęp: luty 2026. 2024. URL: https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/.

[3] Amazon Web Services. *Boto3 Documentation*. Dostęp: luty 2026. 2024. URL: https://boto3.amazonaws.com/v1/documentation/api/latest/index.html.

[4] Amazon Web Services. *Shared Responsibility Model*. Dostęp: luty 2026. 2024. URL: https://aws.amazon.com/compliance/shared-responsibility-model/.

[5] Amazon Web Services. *Signing AWS API Requests*. Dostęp: luty 2026. 2024. URL: https://docs.aws.amazon.com/general/latest/gr/signing_aws_api_requests.html.

[6] Amazon Web Services. *AWS Security Hub Pricing*. Dostęp: luty 2026. 2026. URL: https://aws.amazon.com/security-hub/pricing/.

[7] American Institute of Certified Public Accountants. *2017 Trust Services Criteria (With Revised Points of Focus – 2022)*. Sprawozdanie techniczne. Dostęp: luty 2026. AICPA, 2017. URL: https://www.aicpa-cima.com/resources/download/2017-trust-services-criteria-with-revised-points-of-focus-2022.

[8] Len Bass, Ingo Weber i Liming Zhu. *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional, 2015. ISBN: 978-0134049847.

[9] Capital One. *Information on the Capital One Cyber Incident*. Dostęp: luty 2026. 2019. URL: https://web.archive.org/web/2020/https://www.capitalone.com/digital/facts2019/.

[10] Center for Internet Security. *CIS Critical Security Controls v8.1*. Sprawozdanie techniczne. Dostęp: luty 2026. Center for Internet Security, 2024. URL: https://learn.cisecurity.org/cis-controls-download.

[11] Center for Internet Security. *CIS Amazon Web Services Foundations Benchmark (v6.0.0)*. Sprawozdanie techniczne. Dostęp: luty 2026; wersja v6.0.0 użyta w pracy. Center for Internet Security, 2026. URL: https://downloads.cisecurity.org/.

[12] Cloud Custodian. *Cloud Custodian – Rules Engine for Cloud Security*. Dostęp: luty 2026. 2024. URL: https://github.com/cloud-custodian/cloud-custodian (dostęp 15.01.2025).

[13] Cloud Native Computing Foundation. *Cloud Native 2024: Approaching a Decade of Code, Cloud, and Change*. Sprawozdanie techniczne. Dostęp: luty 2026. CNCF, 2025. URL: https://www.cncf.io/reports/cncf-annual-survey-2024/.

[14] Cloud Security Alliance. *Top Threats to Cloud Computing 2024*. Sprawozdanie techniczne. Dostęp: luty 2026. Cloud Security Alliance, 2024. URL: https://cloudsecurityalliance.org/artifacts/top-threats-to-cloud-computing-2024.

[15] European Union Agency for Cybersecurity (ENISA). *ENISA Threat Landscape 2024*. Sprawozdanie techniczne. Dostęp: luty 2026. ENISA, 2024. URL: https://www.enisa.europa.eu/publications/enisa-threat-landscape-2024.

[16] Diogo A. B. Fernandes, Liliana F. B. Soares, João V. Gomes, Mário M. Freire i Pedro R. M. Inácio. „Security issues in cloud environments: a survey". W: *International Journal of Information Security* 13.2 (2014), s. 113–170. DOI: 10.1007/s10207-013-0208-7.

[17] Forbes. *Wiz Company Profile – Cloud 100*. Dostęp: luty 2026. 2024. URL: https://www.forbes.com/companies/wiz/.

[18] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. ISBN: 978-0321127426.

[19] Erich Gamma, Richard Helm, Ralph Johnson i John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994. ISBN: 978-0201633610.

[20] GitHub. *GitHub Actions Documentation*. Dostęp: luty 2026. 2024. URL: https://docs.github.com/en/actions.

[21] Jez Humble i David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010. ISBN: 978-0321601919.

[22] IBM X-Force. *IBM X-Force 2025 Threat Intelligence Index*. Sprawozdanie techniczne. Dostęp: luty 2026. IBM, 2025. URL: https://www.ibm.com/reports/threat-intelligence.

[23] IEEE. *IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications*. Institute of Electrical i Electronics Engineers, 1998. DOI: 10.1109/IEEESTD.1998.88286.

[24] Microsoft. *What is Microsoft Defender for Cloud?* Dostęp: luty 2026. 2026. URL: https://learn.microsoft.com/en-us/azure/defender-for-cloud/defender-for-cloud-introduction.

[25] Chirag Modi, Dhiren Patel, Bhavesh Borisaniya, Avi Patel i Muttukrishnan Rajarajan. „A survey on security issues and solutions at different layers of Cloud computing". W: *The Journal of Supercomputing* 63.2 (2013), s. 561–592. DOI: 10.1007/s11227-012-0831-5.

[26] National Institute of Standards and Technology. *Framework for Improving Critical Infrastructure Cybersecurity, Version 1.1*. NIST Cybersecurity Framework. NIST, 2018. DOI: 10.6028/NIST.CSWP.04162018.

[27] NCC Group. *ScoutSuite - Multi-Cloud Security Auditing Tool*. Dostęp: luty 2026. 2024. URL: https://github.com/nccgroup/ScoutSuite.

[28] Office of the Comptroller of the Currency. *OCC Assesses $80 Million Civil Money Penalty Against Capital One*. Dostęp: luty 2026. 2020. URL: https://www.occ.gov/news-issuances/news-releases/2020/nr-occ-2020-101.html.

[29] PCI Security Standards Council. *PCI Data Security Standard v4.0*. Sprawozdanie techniczne. Dostęp: luty 2026. PCI SSC, 2022. URL: https://www.pcisecuritystandards.org/document_library/?document=pci_dss&version=4_0.

[30] Prowler. *Prowler - AWS Security Best Practices Assessment*. Dostęp: luty 2026. 2024. URL: https://github.com/prowler-cloud/prowler.

[31] S. Subashini i V. Kavitha. „A survey on security issues in service delivery models of cloud computing". W: *Journal of Network and Computer Applications* 34.1 (2011), s. 1–11. DOI: 10.1016/j.jnca.2010.07.006.

[32] Synergy Research Group. *Cloud Market Share Trends: Big Three Together Hold 63% While Oracle and the Neoclouds Inch Higher*. Dostęp: styczeń 2026. 2025. URL: https://www.srgresearch.com/articles/cloud-market-share-trends-big-three-together-hold-63-while-oracle-and-the-neoclouds-inch-higher.

[33] U.S. Department of Health and Human Services. *HIPAA Security Rule Compliance Guidelines*. Sprawozdanie techniczne. Dostęp: luty 2026. HHS Office for Civil Rights, 2013. URL: https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/.

[34] Dimitrios Zissis i Dimitrios Lekkas. „Addressing cloud computing security issues". W: *Future Generation Computer Systems* 28.3 (2012), s. 583–592. DOI: 10.1016/j.future.2010.12.006.

SPIS RYSUNKÓW

1.1 Kontekst problemu Cloud Security Posture Management 11
2.1 Usługi AWS objęte skanowaniem przez platformę 20
2.2 Pipeline CI/CD platformy AWS Infrastructure Intelligence 21
3.1 Architektura wysokopoziomowa platformy AWS Infrastructure Intelligence 25
3.2 Diagram komponentów platformy 27
3.3 Diagram sekwencji UML procesu skanowania 28
4.1 Diagram klas warstwy kolekcji 35
4.2 Rozkład znalezisk według poziomu krytyczności 43
4.3 Porównanie czasu skanowania: tryb mock vs produkcyjny (estymacja) . . 43

SPIS TABEL

2.1 Porównanie wybranych narzędzi CSPM 14
2.2 Porównanie frameworków zgodności bezpieczeństwa 17
2.3 Mapowanie frameworków zgodności na domeny kontroli technicznych AWS 17
3.1 Macierz wymagań funkcjonalnych i niefunkcjonalnych 24
3.2 Stack technologiczny platformy z uzasadnieniem wyboru 31
3.3 Podsumowanie środowiska testowego 33
4.1 Charakterystyka kolektorów usług AWS 35
4.2 Fragment mapowania reguł CIS na frameworki zgodności 37
4.3 Podsumowanie wykonawcze raportu Excel – wyniki skanowania testowego 38
4.4 Etapy pipeline'u CI/CD z czasami wykonania 40
4.5 Wyniki skanowania środowiska testowego 42
4.6 Napotkane wyzwania techniczne i ich rozwiązania 46
5.1 Weryfikacja realizacji celów pracy 48
B.1 Niezgodności IAM wprowadzone w środowisku testowym 64
B.2 Niezgodności Storage wprowadzone w środowisku testowym 65
B.3 Niezgodności Compute wprowadzone w środowisku testowym 65
B.4 Niezgodności Networking wprowadzone w środowisku testowym 66
B.5 Niezgodności Logging wprowadzone w środowisku testowym 66

SPIS LISTINGÓW

2.1 Użycie boto3 do listowania EC2 20
3.1 Format wyniku kolektora 27
3.2 Wzorzec architektury dwutrybowej 28
3.3 Schema JSON dla zasobu AWS 29
3.4 Struktura wyników analizy 29
3.5 Mapowanie reguł na frameworki 30
3.6 Główne zależności projektu 32
4.1 Implementacja klasy BaseCollector 34
4.2 Struktura ExcelReporter (fragment, uproszczony) 38
4.3 Endpointy REST API 39
4.4 Struktura przepływu pracy CI/CD 40
4.5 Test jednostkowy kolektora EC2 41
4.6 Test reguły CIS-2.2.1 (szyfrowanie EBS) 42
4.7 Dekorator mapowania reguły 44
4.8 Wykładnicze opóźnienie 45
4.9 Czcionki Unicode 46
A.1 BaseCollector – definicja klasy i konstruktor 59
A.2 BaseCollector – metoda collect() (cd.) 60
A.3 BaseCollector – obsługa throttlingu AWS (cd.) 61
A.4 ComplianceAnalyzer – struktura i metoda analyze() 61
A.5 ComplianceAnalyzer – przykładowe reguły CIS (cd.) 62
A.6 Struktura projektu – moduł src/ 62
A.7 Struktura projektu – analizatory i reportery (cd.) 63
A.8 Struktura projektu – testy i konfiguracja (cd.) 63

ZAŁĄCZNIK A

WYBRANE FRAGMENTY KODU ŹRÓDŁOWEGO

Załącznik zawiera pełne implementacje wybranych klas platformy AWS Infrastructure Intelligence. Fragmenty kodu uzupełniają skrócone listingi z rozdziału 4 i pozwalają przeanalizować zastosowane wzorce projektowe.

A.1 Klasa BaseCollector

Pełna implementacja abstrakcyjnej klasy bazowej dla kolektorów usług AWS. Klasa definiuje wzorzec Template Method (sekcja 4.1.1). Ze względu na objętość, kod podzielono na trzy fragmenty.

```
from abc import ABC, abstractmethod
from typing import Any, Dict, List, Optional
import time, random
from loguru import logger

class BaseCollector(ABC):
    """Abstrakcyjna klasa bazowa dla kolektorow uslug AWS."""

    def __init__(self, session, config: Optional[Dict] = None):
        self.session = session
        self.config = config or {}
        self.service_name = self._get_service_name()

    @abstractmethod
    def _get_service_name(self) -> str:
        pass

    @abstractmethod
    def _collect_resources(self, client, region: str) -> List[Dict
    ]:
        pass
```

Listing A.1. BaseCollector – definicja klasy i konstruktor

Źródło: Opracowanie własne na podstawie `src/collectors/base_collector.py`

```
    def collect(self, regions: List[str]) -> Dict[str, Any]:
        """Glowna metoda kolekcji - wzorzec Template Method."""
        all_resources, by_region, errors = [], {}, []
        start_time = time.time()

        for region in regions:
            try:
                client = self.session.get_client(self.service_name,
    region)
                resources = self._collect_resources(client, region)
                for r in resources:
                    r["region"] = region
                all_resources.extend(resources)
                by_region[region] = resources
            except Exception as e:
                errors.append({"region": region, "error": str(e)})

        return {"resources": all_resources, "by_region": by_region,
                "metadata": {"service": self.service_name,
                             "total_count": len(all_resources),
                             "duration": round(time.time() -
    start_time, 2)}}
```

Listing A.2. BaseCollector – metoda collect() (cd.)

Źródło: Opracowanie własne na podstawie `src/collectors/base_collector.py`

```
    def _retry_api_call(self, client, method: str,
                        max_retries: int = 5, **kwargs) -> Dict:
        """Wywolanie API z wykladniczym opoznieniem (backoff)."""
        func = getattr(client, method)
        for attempt in range(max_retries):
            try:
                return func(**kwargs)
            except client.exceptions.ClientError as e:
                if "Throttling" in str(e) and attempt < max_retries
    - 1:
                    delay = (2 ** attempt) + random.uniform(0, 0.5)
                    time.sleep(delay)
                else:
                    raise
```

Listing A.3. BaseCollector – obsługa throttlingu AWS (cd.)

Źródło: Opracowanie własne na podstawie `src/collectors/base_collector.py`

A.2 Fragment ComplianceAnalyzer – reguły CIS

Poniżej przedstawiono fragment klasy `ComplianceAnalyzer` z implementacją wybranych reguł CIS AWS Foundations Benchmark. Identyfikatory CIS w listingach odpowiadają mapowaniu użytemu w implementacji narzędzia.

```
class ComplianceAnalyzer:
    """Analizator zgodnosci z CIS AWS Foundations Benchmark."""

    def __init__(self):
        self.rules = self._register_rules()
        self.findings = []

    def analyze(self, collected_data: Dict) -> ComplianceResult:
        for rule in self.rules:
            service_data = collected_data.get(rule.service, {})
            for resource in service_data.get("resources", []):
                if not rule.check_fn(resource):
                    self.findings.append(ComplianceFinding(
                        rule_id=rule.rule_id,
                        severity=rule.severity,
                        resource_id=resource["resource_id"],
                        remediation=rule.remediation))
        return self._calculate_result()
```

Listing A.4. ComplianceAnalyzer – struktura i metoda analyze()

Źródło: Opracowanie własne na podstawie `src/analyzers/compliance_analyzer.py`

```
    @compliance_rule("CIS-1.4", severity=Severity.CRITICAL,
        mappings={"soc2": ["CC6.1"], "hipaa": ["164.312(d)"]})
    def check_no_root_access_keys(self, resource: Dict) -> bool:
        """CIS 1.4: Brak kluczy dostepu dla konta root."""
        if resource.get("user_name") == "<root_account>":
            return not resource.get("access_key_active", False)
        return True

    @compliance_rule("CIS-2.2.1", severity=Severity.HIGH,
        mappings={"soc2": ["CC6.1"], "pci_dss": ["3.4.1"]})
    def check_ebs_encryption(self, resource: Dict) -> bool:
        """CIS 2.2.1: Szyfrowanie EBS wlaczone."""
        return resource.get("properties", {}).get("encrypted",
    False)
```

Listing A.5. ComplianceAnalyzer – przykładowe reguły CIS (cd.)

Źródło: Opracowanie własne na podstawie `src/analyzers/compliance_analyzer.py`

A.3 Struktura katalogów projektu

Poniższe listingi przedstawiają pełną strukturę katalogów projektu AWS Infrastructure Intelligence.

```
aws-infrastructure-intelligence/
|-- src/
|   |-- __init__.py
|   |-- main.py                    # InfrastructureScanner
|   |-- cli.py                     # Interfejs CLI (Click)
|   |-- api/
|   |   |-- main.py                # REST API (FastAPI)
|   |-- collectors/
|   |   |-- base_collector.py      # Klasa abstrakcyjna (345 LOC)
|   |   |-- ec2_collector.py       # 248 LOC
|   |   |-- ebs_collector.py       # 242 LOC
|   |   |-- s3_collector.py        # 385 LOC
|   |   |-- rds_collector.py       # 312 LOC
|   |   |-- vpc_collector.py       # 308 LOC
|   |   |-- iam_collector.py       # 359 LOC
|   |   |-- lambda_collector.py    # 224 LOC
|   |   |-- security_group_collector.py # 356 LOC
|   |   |-- cost_collector.py      # 255 LOC
```

Listing A.6. Struktura projektu – moduł src/

Źródło: Opracowanie własne

```
|   |-- analyzers/
|   |   |-- compliance_analyzer.py # 2535 LOC (rdzen systemu)
|   |   |-- security_analyzer.py   # 598 LOC
|   |   |-- cost_optimizer.py      # 474 LOC
|   |   |-- frameworks/
|   |       |-- soc2.py            # 561 LOC
|   |       |-- hipaa.py           # 523 LOC
|   |       |-- pci_dss.py         # 653 LOC
|   |-- reporters/
|   |   |-- excel_reporter.py      # Raporty xlsx (11 arkuszy)
|   |   |-- pdf_reporter.py        # Raporty PDF
|   |   |-- html_reporter.py       # Raporty HTML
|   |   |-- json_reporter.py       # Raporty JSON
|   |   |-- chart_generator.py     # Wykresy matplotlib
|   |-- utils/
|       |-- aws_session.py         # AWSSessionManager + MockClient
|       |-- config_loader.py       # Parsowanie YAML
|       |-- logger.py              # Konfiguracja loguru
```

Listing A.7. Struktura projektu – analizatory i reportery (cd.)

Źródło: Opracowanie własne

```
|-- tests/
|   |-- test_collectors.py         # 27 testow
|   |-- test_analyzers.py          # 32 testy
|   |-- test_reporters.py          # 28 testow
|-- data/mock/                     # 9 plikow JSON (dane testowe)
|-- .github/workflows/ci.yml       # Pipeline CI/CD (6 etapow)
|-- Dockerfile                     # Wieloetapowe budowanie
|-- config.yaml                    # Konfiguracja skanowania
|-- requirements.txt               # 28 zaleznosci
```

Listing A.8. Struktura projektu – testy i konfiguracja (cd.)

Źródło: Opracowanie własne

ZAŁĄCZNIK B

PRZYKŁADOWA LISTA WYKRYTYCH NIEZGODNOŚCI

Załącznik prezentuje 42 z 73 niezgodności wykrytych w trybie mock, pogrupowanych w pięciu domenach bezpieczeństwa (IAM, Storage, Compute, Networking, Logging). Pełna lista znajduje się w raporcie JSON dostępnym w repozytorium projektu. Identyfikatory CIS w tabelach odpowiadają mapowaniu użytemu w implementacji narzędzia.

B.1 Niezgodności w domenie IAM

Tab. B.1. Niezgodności IAM wprowadzone w środowisku testowym

Nr	CIS ID	Opis niezgodności	Krytyczność
1	1.4	Aktywny klucz dostępu dla konta root	Krytyczny
2	1.5	Brak MFA dla konta root	Krytyczny
3	1.8	Polityka haseł IAM bez wymagania wielkiej litery	Wysoki
4	1.10	Użytkownik IAM bez MFA z aktywnym dostępem	Średni
5	1.12	Klucz dostępu starszy niż 90 dni (użytkownik test-user-1)	Średni
6	1.13	Użytkownik z dwoma aktywnymi kluczami dostępu	Niski
7	1.16	Polityka IAM z uprawnieniem * (full admin)	Informacyjny
8	1.17	Rola IAM bez ograniczenia Principal	Informacyjny

Źródło: Opracowanie własne

B.2 Niezgodności w domenie Storage

Tab. B.2. Niezgodności Storage wprowadzone w środowisku testowym

Nr	CIS ID	Opis niezgodności	Krytyczność
9	2.1.2	Bucket S3 bez szyfrowania server-side (bucket-test-1)	Wysoki
10	2.1.2	Bucket S3 bez szyfrowania server-side (bucket-test-2)	Wysoki
11	–	Bucket S3 z wyłączonym logowaniem dostępu	Średni
12	2.1.3	Bucket S3 bez wersjonowania (bucket-logs)	Średni
13	2.1.5	Bucket S3 z publicznym ACL (bucket-public-test)	Wysoki
14	2.1.5	Brak blokady publicznego dostępu na poziomie konta	Średni
15	2.2.1	Wolumen EBS bez szyfrowania (vol-001)	Wysoki
16	2.2.1	Wolumen EBS bez szyfrowania (vol-002)	Niski
17	2.2.1	Wolumen EBS bez szyfrowania (vol-003)	Niski
18	2.2.1	Wolumen EBS bez szyfrowania (vol-004)	Wysoki
19	2.2.1	Wolumen EBS bez szyfrowania (vol-005)	Wysoki
20	2.3.1	Instancja RDS bez szyfrowania at-rest	Wysoki
21	2.3.2	Instancja RDS z publiczną dostępnością	Średni
22	–	Bucket S3 bez polityki cyklu życia	Informacyjny
23	–	Brak automatycznych snapshotów EBS	Niski

Źródło: Opracowanie własne

B.3 Niezgodności w domenie Compute

Tab. B.3. Niezgodności Compute wprowadzone w środowisku testowym

Nr	CIS ID	Opis niezgodności	Krytyczność
24	–	Instancja EC2 bez detailed monitoring (i-001)	Niski
25	–	Instancja EC2 bez detailed monitoring (i-002)	Niski
26	–	Instancja EC2 bez detailed monitoring (i-003)	Niski
27	–	Instancja EC2 z przestarzałym AMI (starszy niż 180 dni)	Niski
28	–	Funkcja Lambda bez konfiguracji VPC	Niski
29	–	Funkcja Lambda z przestarzałym runtime (Python 3.7)	Niski
30	–	Instancja EC2 bez przypisanego profilu IAM	Informacyjny

Źródło: Opracowanie własne

B.4 Niezgodności w domenie Networking

Tab. B.4. Niezgodności Networking wprowadzone w środowisku testowym

Nr	CIS ID	Opis niezgodności	Krytyczność
31	5.2	Security Group z otwartym SSH (0.0.0.0/0:22)	Wysoki
32	5.3	Security Group z otwartym RDP (0.0.0.0/0:3389)	Wysoki
33	5.4	Domyślna Security Group z regułą ingress	Niski
34	3.9	VPC bez włączonych Flow Logs (vpc-001)	Średni
35	–	Subnet publiczny z routingiem do IGW bez NAT	Niski
36	–	Security Group z nadmiarową regułą (duplikat)	Niski
37	–	VPC z domyślnym CIDR (172.31.0.0/16)	Niski
38	–	Brak Network ACL dla subnetu prywatnego	Niski
39	–	Security Group z otwartym HTTPS (0.0.0.0/0:443)	Informacyjny

Źródło: Opracowanie własne

B.5 Niezgodności w domenie Logging

Tab. B.5. Niezgodności Logging wprowadzone w środowisku testowym

Nr	CIS ID	Opis niezgodności	Krytyczność
40	3.1	CloudTrail wyłączony w regionie eu-west-1	Średni
41	3.4	CloudTrail bez walidacji integralności logów	Średni
42	3.7	Logi CloudTrail bez szyfrowania KMS	Średni

Źródło: Opracowanie własne

B.6 Podsumowanie

Przedstawiono 42 przykładowe niezgodności (spośród łącznie 73 wykrytych). Rozkład krytyczności w zaprezentowanym podzbiorze:

- Krytyczny: 2 (4,8% podzbioru)
- Wysoki: 10 (23,8% podzbioru)
- Średni: 10 (23,8% podzbioru)
- Niski: 15 (35,7% podzbioru)
- Informacyjny: 5 (11,9% podzbioru)

Powyższy rozkład dotyczy wyłącznie 42 wybranych przykładów. Rozkład krytyczności dla pełnego zestawu 73 niezgodności przedstawiono w tabeli 4.5 i na rysunku 4.2 w rozdziale 4.